



西安电子科技大学
XIDIAN UNIVERSITY

Origami

Yulong Shen, Ke Cheng, Anxiao Song, Xiangxi Liu, Xianzhi Shi,
Junxian Wu, Yangjing Wu, Xutong Mu, Lele Zheng, Zhenxing Niu

Organizations: Xidian University

June 26, 2026

Abstract

Origami is a multivariate quadratic signature design derived from the Oil-and-Vinegar paradigm and motivated by the need to reduce the attack surface created by globally visible algebraic structure. The main idea is to preserve the structure needed for efficient signing inside the secret key, while publishing only a flattened and externally mixed MQ representation over the base field. Concretely, Origami combines local partitioned hidden structure, ordinary-MQ public compilation, and dense input/output hiding so that the signer retains a blockwise linear solving method but the verifier sees only a standard public MQ map.

This report presents a full written specification of Origami, including parameters, hashing and serialization rules, key generation, signing, verification, and a formal correctness proof. It also develops a security analysis based on hidden-structure indistinguishability and multi-target MQ inversion hardness, proposes candidate parameter families for several classical and quantum security targets, and reports implementation results for reference, optimized, and AVX2 implementations. The experimental results show that Origami keeps secret keys and signatures compact, while the main scalability bottleneck remains the size and generation cost of the explicit public MQ key.

Contents

1	Introduction	3
1.1	Background	3
1.2	The Basic Idea	4
1.3	Description of Differences	6
1.4	Algorithmic Characteristics	7
2	The Origami Protocol Specification	8
2.1	Written Specification	8
2.2	Parameters	8
2.3	Preliminaries and Notation	11
2.4	Hashing and randomness expansion	14
2.4.1	Hash Function	14
2.4.2	Randomness Expansion	15
2.4.3	Hashing to Field Vectors	16
2.5	Data types and conversions	16
2.5.1	Byte Strings	17
2.5.2	Base-Field Vectors	17
2.5.3	Hidden Local Representations	17
2.5.4	Generic Vector Encoding	18
2.5.5	Encoding Hidden Local Objects and Public Coefficients	18
2.5.6	Flattening Convention	19
2.5.7	Flattening and Signature Encoding	20
2.5.8	Public-Key and Secret-Key Serialization	20
2.5.9	Encoding Requirement	21
2.6	The Basic Origami functionalities	22
2.6.1	Key Generation Algorithm	22
2.6.2	Sign Algorithm	24
2.6.3	Verify Algorithm	26
2.6.4	Correctness	27
3	Detailed performance analysis	30
3.1	Implementation Setup	30
3.2	Parameter Sets and Key Sizes	30
3.3	Communication and Storage Cost	32
3.4	Implementation performance	34
3.5	Restart behavior performance	36
4	Security Analysis	37
4.1	Hard Problems	40
4.2	Security Proof	42
4.3	Discussion of the Advantage Loss in the Security Proof	46
4.4	Analysis of Known Attacks	46
4.5	Signing Failure Probability and Its Security Impact	53

1 Introduction

1.1 Background

Multivariate quadratic (MQ) cryptography is a longstanding approach to post-quantum public-key cryptography, rooted in the hardness of solving large systems of quadratic equations over finite fields. A public MQ map is written as

$$P = (p_1, \dots, p_m) : \mathbb{F}_q^n \longrightarrow \mathbb{F}_q^m,$$

where each p_i is a quadratic polynomial over $\mathbb{F}_q$. Unbalanced Oil-and-Vinegar (UOV) signatures capture both the appeal and the difficulty of this approach.

In a UOV-type trapdoor, the secret coordinates are split into a vinegar part and an oil part,

$$\mathbb{F}_q^n = V \oplus O,$$

and the central map is chosen so that no quadratic oil-oil terms appear. Equivalently, for oil vectors $o \in O$, the quadratic part vanishes in the oil direction. If the polar form of P is

$$P'(x, y) = P(x + y) - P(x) - P(y),$$

then, for a signing vector $s = v + o$ with $v \in V$ and $o \in O$, one has

$$P(v + o) = P(v) + P'(v, o).$$

Thus, after the vinegar variables are fixed, the unknown oil variables appear only linearly. For a target $t \in \mathbb{F}_q^m$, signing reduces to a linear system of the form

$$A(v)o = t - D(v).$$

This linearization explains the efficiency of UOV-type signatures. However, the same hidden oil-vinegar decomposition may leave global traces in the public quadratic forms. Therefore, the central challenge for UOV-type signatures is not only to build an efficient signer, but also to publish a verification key that does not expose an algebraic object equivalent to the signing trapdoor.

This tension has largely shaped the security history of UOV and its variants. The UOV construction shows that a hidden oil space can provide an efficient trapdoor for multivariate signatures, but the same subspace becomes a security weakness once it, or sufficient information about it, is recoverable from the public key. Subsequent attacks showed that this leakage may appear implicitly through polar forms, linear-algebraic relations, and MinRank-type rank constraints. The wedge attack is a particularly serious recent example: over fields of characteristic two, the public polar forms of UOV are alternating and can be interpreted in the exterior algebra, where they impose linear relations on the Plücker embedding of the dual oil space. Solving the resulting sparse linear system can then recover the hidden oil space.

Although these attacks differ technically, they point to a common design lesson. A UOV-like public key becomes fragile when information from many public quadratic forms can be aligned toward a single coherent hidden object, such as an oil space, a dual relation, a low-rank structure, or an equivalent global invariant. This observation suggests that

hidden structure need not be abandoned altogether. Rather, the structure used for signing should be made local and non-aligning: it should remain available to the legitimate signer in secret coordinates, while its public traces should not combine into one globally exploitable target.

Origami is designed around this principle. Rather than relying on a single global oil space, Origami distributes the signing trapdoor across local zones. Each zone has its own hidden local representations and admits a linear-solving step: after the signer fixes the current vinegar variables and a privately sampled part of the oil coordinates, the remaining coordinates are obtained by solving a local linear system for the target zone derived from the message and salt. Controlled cross-zone terms preserve this linearity for the signer while preventing the public map from appearing as a collection of independent UOV instances.

Origami stores a compressed public key that expands to a standard MQ map over the base field. More precisely, the compressed public description contains a public expansion seed together with an explicit residual coefficient vector, and verification is performed after reconstructing the corresponding expanded public MQ map. Thus, the verifier evaluates an ordinary system of quadratic equations, whereas the signer retains the hidden local representations and private oil-coordinate partitions needed for sequential solving. The design goal is to keep the exploitable structure local to the secret key, while exposing only a flattened, cross-zone-coupled, and coefficient-mixed MQ representation that does not explicitly reveal a global oil-vinegar decomposition.

1.2 The Basic Idea

The basic idea of Origami is to divide the signing trapdoor into several local zones while keeping the public interface as an ordinary MQ map over the base field. In the secret key, each zone is described by hidden local representations and a private oil-coordinate partition. The signer fixes the current vinegar variables, samples the prescribed part of the oil coordinates, and solves the remaining oil coordinates by a local linear system. In the public key, these local structures are compiled into an expanded public MQ map reconstructed from a compressed public description, so that verification remains standard MQ evaluation.

This local organization is used to meet two requirements simultaneously. First, the secret description must keep enough structure for efficient signing: after the signer has fixed the previously solved zones, the current vinegar variables, and the sampled oil coordinates, each stage reduces to a local linear system. Second, the public description should not expose a single global object, such as a common oil space, a common vinegar space, or a uniform matrix algebra, from which the hidden structure could be recovered. For this reason, Origami keeps the signable structure inside local zones and uses cross-zone coupling and coefficient mixing so that the public MQ map does not provide a coherent target for reconciliation, intersection, MinRank, or wedge-style attacks.

After the local signable structure has been fixed, Origami compiles it into an ordinary MQ map over the base field $\mathbb{F}_q$. The verifier therefore does not work with the hidden local algebras, the private oil-coordinate partitions, or the sequential solving order. Instead, the verifier only evaluates the expanded public MQ map reconstructed from the compressed public description. Concretely, the compressed public key contains a public expansion

seed and an explicit residual coefficient vector,

$$pk = (\text{params}, \text{seed}_{pk}, R_{pk}).$$

The public expansion seed determines the seed-generated part of the public coefficient array, while R_{pk} supplies the residual coefficients needed to obtain the final signable flattened map. Thus, verification is performed after reconstructing

$$P_{\text{pub}} : \mathbb{F}_q^N \longrightarrow \mathbb{F}_q^M$$

from this compressed public description.

The residual coefficient positions are deliberately mixed across several term classes, rather than being aligned with one public class of sampled oil coordinates, solved oil coordinates, or vinegar coordinates. Consequently, the public MQ map remains a standard base-field MQ representation, while the local partitions and zone boundaries remain available only to the signer.

The signing procedure follows the same separation. The target vector derived from the message and salt is split into zone targets. The signer then processes the zones in order. In zone j , the previously solved zones are already fixed; the signer samples the current vinegar variables and the sampled part of the oil coordinates, and then solves the remaining oil coordinates from a local linear system. If the corresponding local matrix does not have full row rank, the signer resamples the current local randomness and tries again.

The above idea can be written more concretely in the following form. For each zone $j \in [d+1]$, let $Z_j \in \mathbb{F}_q^{k_j o_j l_j^2}$ be the flattened oil-coordinate vector of that zone. The secret key contains a private oil-coordinate partition

$$\rho_j : Z_j \longmapsto (U_j, W_j), \quad U_j \in \mathbb{F}_q^{a_j}, \quad W_j \in \mathbb{F}_q^{b_j}.$$

The coordinates in U_j are sampled, while the coordinates in W_j are solved. Before zone j is solved, the previous zones, the current vinegar variables, and U_j have already been fixed. We write these known variables as

$$X_j = \left(Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j \right).$$

The local zone map is required to be affine in the unknown coordinates W_j :

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j,$$

where

$$D_j(X_j) \in \mathbb{F}_q^{M_j}, \quad A_j(X_j) \in \mathbb{F}_q^{M_j \times b_j}.$$

Thus, for a target zone $u^{(j)} \in \mathbb{F}_q^{M_j}$, signing in zone j reduces to the local linear system

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j).$$

If $A_j(X_j)$ has full row rank, the signer samples a solution from the corresponding affine solution space and then recovers

$$Z_j = \rho_j^{-1}(U_j, W_j).$$

After all zones have been solved, the internal assignment is flattened into a public signature vector. The verifier does not reconstruct the private partitions or the local hidden representations. Instead, it expands the compressed public key and evaluates P_{pub} .

1.3 Description of Differences

The Description of Differences of Origami are summarized as follows.

First, Origami replaces the single global oil-space trapdoor with a local-zone signing trapdoor. The variables are divided into several local zones, and each zone has its own hidden local representations. Signing is no longer performed by solving one global oil-coordinate system. Instead, the signer processes the zones sequentially and solves one local linear system at each stage.

Second, Origami introduces a private sampled/solved split of the oil coordinates in each zone. For zone j , the flattened oil-coordinate vector Z_j is privately partitioned as

$$\rho_j : Z_j \mapsto (U_j, W_j),$$

where U_j is sampled and W_j is solved. After the previous zones, the current vinegar variables, and U_j have been fixed, the zone equation has the form

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j.$$

Thus, for a target block $u^{(j)}$, signing in zone j reduces to

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j).$$

This preserves the UOV-type linear solving mechanism, but attaches it to private local partitions rather than to one public global oil space.

Third, Origami adds controlled cross-zone coupling between different local zones. These cross-zone terms are chosen so that they do not introduce quadratic terms in the solved coordinates W_j , and therefore do not destroy the local linear system used for signing. At the same time, they prevent the public map from decomposing into a direct product of independent UOV instances.

Fourth, Origami changes the public-key generation procedure from explicit dense publication to seed-based compressed publication. The public key is stored as

$$pk = (\text{params}, \text{seed}_{pk}, R_{pk}),$$

where seed_{pk} generates the seed-derived part of the public coefficient array, and R_{pk} contains the explicit residual coefficients. During verification, the verifier expands this compressed description into the corresponding public MQ map

$$P_{\text{pub}} : \mathbb{F}_q^N \longrightarrow \mathbb{F}_q^M$$

and checks the standard MQ evaluation equation.

Fifth, Origami mixes the residual coefficient positions across several term classes. The residual coefficients are not placed only on one visible class of oil-related or vinegar-related terms. This prevents the sampled coordinates, solved coordinates, and local zone boundaries from appearing as a public template, while still allowing the expanded map to match the signable internal construction.

1.4 Algorithmic Characteristics

The algorithmic characteristics of Origami can be summarized from six aspects: novelty, simplicity, flexibility, compatibility, scalability, and implementation portability. These characteristics are based on the concrete design and implementation reported in this work, rather than on unsupported platform-wide performance claims.

1. **Novelty.** Origami replaces the conventional single global oil-vinegar trapdoor with a local-zone signing trapdoor. Each local zone has its own hidden local representations, hidden local algebras, and private oil-coordinate partitions. The signer performs local sequential solving, while the public representation does not expose one globally aligned oil space. This is the main structural difference between Origami and conventional global oil-vinegar constructions.
2. **Simplicity.** Origami keeps the public verification interface simple. Although the signer uses hidden local structures internally, the verifier only reconstructs and evaluates a standard MQ map over the base field. The verifier does not need to recover the hidden local algebras, private oil-coordinate partitions, sampled/solved splits, or sequential solving order.
3. **Flexibility.** Origami uses a private sampled/solved split inside each local zone. During signing, the signer samples part of the local coordinates and solves the remaining coordinates by a local linear system. This design allows the local-zone parameters, the number of local copies, and the sampled/solved dimensions to be adjusted according to the target security level and implementation requirement.
4. **Scalability.** Origami supports multiple security levels by scaling the number of local zones, the number of local copies, the public input and output dimensions, and the residual coefficient positions. The submitted instances follow the same algorithmic framework while targeting different classical and quantum security levels.
5. **Implementation measurability and platform portability.** Origami provides both a portable ISO C reference implementation and a general 64-bit optimized implementation. These two paths use the same public API, key format, signing workflow, and verification workflow, which makes the algorithmic cost measurable under the same experimental platform. The reported results should be understood as portable C and general 64-bit PC software results. They do not claim a uniform performance advantage across different hardware platforms, and they do not rely on AVX2 or SIMD-specific instructions.

2 The Origami Protocol Specification

2.1 Written Specification

This section presents the formal protocol specification of Origami. The fundamental design principle of Origami is to keep the algebraic structure needed for signing inside the secret key, while publishing only a compressed description of a multivariate quadratic map over the base field. The signer uses hidden local representations and private coordinate partitions to perform sequential linear solving, whereas the verifier reconstructs the public MQ map from the compressed public key and checks an ordinary MQ evaluation equation.

Specifically, the structures essential for efficient inversion—namely the hidden local algebras, the private oil-coordinate partitions, the sampled/solved split of local oil coordinates, and the prescribed sequential solving order—are strictly confined within the secret key. By contrast, at the interface level the public key is a compressed description consisting of the public parameters, a derived public expansion seed, and an explicit residual coefficient vector. The public expansion seed is derived from the master secret expansion seed during key generation and is then published as part of the compressed public key. The verifier expands this compressed description into the corresponding multivariate quadratic map over the base field $\mathbb{F}_q$, and does not need to know the hidden local algebras, the private partitions, the sampled/solved split, or the signable internal representation.

To implement this principle, Origami is organized into a three-layer architectural framework:

- **Hidden Local Structure Layer:** A sequentially solvable central map is constructed in secret local coordinates. Each local zone uses hidden local representations, hidden local algebras, private oil-coordinate partitions, and a sampled/solved split of local oil coordinates, so that signing reduces to sequential linear solving.
- **Public MQ Compilation Layer:** The internal signable map is flattened into a multivariate quadratic map over the base field $\mathbb{F}_q$. The public MQ map does not publish the hidden local representations, the hidden local algebras, or the private oil-coordinate partitions.
- **Compressed Public Key Layer:** The compressed public key consists of the public parameters, the derived public expansion seed, and an explicit residual coefficient vector. During verification, the public expansion seed generates the seed-defined part of the public MQ map, while the residual coefficient vector supplies the explicit coefficients on the public residual-position set. This residual-position set is determined by the public parameters and the derived public expansion seed, and it does not reveal the private oil-coordinate partitions or the sampled/solved split.

2.2 Parameters

The Origami signature algorithm is parameterized by the following values:

- q , the size of the base field $\mathbb{F}_q$.
- d , the number of main local zones. Origami also uses one tail zone, indexed by $d+1$.
- For each local zone $j \in 1, \dots, d+1$:

- k_j , the number of local copies in the j -th local zone.
- v_j , the vinegar dimension of one local copy in the j -th local zone.
- o_j , the oil dimension of one local copy in the j -th local zone.
- m_j , the number of hidden-algebra output coordinates in the j -th local zone.
- l_j , the matrix dimension of the hidden local algebra in the j -th local zone.
- a_j , b_j , and δ_j , the parameters that determine the private sampled/solved split of the flattened oil coordinates in the j -th local zone.
- κ_{sk} , the bit length of the master secret expansion seed.
- κ_{pk} , the bit length of the public expansion seed derived from the master secret expansion seed.
- λ_{salt} , the bit length of the signing salt.
- H , the extendable-output hash function used for hashing and seed expansion.

For each local zone $j \in \{1, \dots, d+1\}$ and each local copy $i \in \{1, \dots, k_j\}$, the secret key contains a hidden local algebra

$$K_{j,i} \subseteq \text{Mat}_{l_j}(\mathbb{F}_q)$$

isomorphic to an extension field of degree l_j . This hidden local algebra is not published explicitly. Instead, it is reconstructed directly from the master secret expansion seed whenever the signer expands the secret local data. More precisely, the signer uses seed_{sk} with the domain-separated labels **embed-S** and **embed-C** to sample a matrix $S_{j,i} \in \text{Mat}_{l_j}(\mathbb{F}_q)$ whose characteristic polynomial is irreducible and an invertible conjugating matrix $C_{j,i} \in \text{GL}_{l_j}(\mathbb{F}_q)$. It then defines

$$T_{j,i} = C_{j,i}^{-1} S_{j,i} C_{j,i}, \quad K_{j,i} = \mathbb{F}_q[T_{j,i}].$$

Because different local copies use independently generated conjugations, the expanded public MQ map does not retain a common aligned basis across all local zones and copies.

For local copy i in local zone j , we write the internal variable block as

$$Y_i^{(j)} = (V_i^{(j)}, O_i^{(j)}) \in K_{j,i}^{v_j} \times K_{j,i}^{o_j}. \quad (1)$$

Here $V_i^{(j)}$ denotes the vinegar part of the i -th local copy, and $O_i^{(j)}$ denotes the oil part of the i -th local copy. For the whole local zone, we write

$$Y^{(j)} = (Y_1^{(j)}, \dots, Y_{k_j}^{(j)}), \quad Y^{(<j)} = (Y^{(1)}, \dots, Y^{(j-1)}). \quad (2)$$

The flattened oil-coordinate vector of local zone j is denoted by

$$Z_j \in \mathbb{F}_q^{k_j o_j l_j^2}. \quad (3)$$

The secret key contains a private oil-coordinate partition

$$\rho_j : Z_j \mapsto (U_j, W_j), \quad U_j \in \mathbb{F}_q^{a_j}, \quad W_j \in \mathbb{F}_q^{b_j}.$$

This partition is generated deterministically from the master secret expansion seed. More precisely, let

$$\pi_j^{\text{oil}} = \text{Permute}(\text{seed}_{\text{sk}}, \text{oil-partition} \parallel \text{param_id} \parallel j, k_j o_j l_j^2)$$

be a secret permutation of the $k_j o_j l_j^2$ flattened oil-coordinate positions in local zone j . If

$$\pi_j^{\text{oil}}(Z_j) = (z_{j,1}, \dots, z_{j,k_j o_j l_j^2}),$$

then

$$U_j = (z_{j,1}, \dots, z_{j,a_j}), \quad W_j = (z_{j,a_j+1}, \dots, z_{j,a_j+b_j}).$$

The inverse map ρ_j^{-1} is obtained by applying the inverse permutation $(\pi_j^{\text{oil}})^{-1}$. Thus, the sampled/solved split is reproducible from seed_{sk} , but it is not published as part of the compressed public key.

During signing, the vinegar variables and the sampled part U_j are sampled, while the solved part W_j is obtained by sequential linear solving. These hidden local representations and private oil-coordinate partitions exist only in the secret local coordinates and are not published in the compressed public key.

After flattening the hidden local algebra objects into coordinates over the base field $\mathbb{F}_q$, each element of $K_{j,i}$ occupies l_j^2 base-field coordinates. Hence, local zone j contributes

$$M_j = m_j l_j^2$$

output coordinates. The total public input dimension and output dimension are

$$N = \sum_{j=1}^{d+1} k_j (v_j + o_j) l_j^2, \quad M = \sum_{j=1}^{d+1} M_j. \quad (4)$$

Equivalently,

$$M = \sum_{j=1}^{d+1} m_j l_j^2. \quad (5)$$

These dimensions determine, respectively, the size of the public signature vector and the size of the public target vector after expansion to the base field $\mathbb{F}_q$.

The private sampled/solved split of local zone j satisfies

$$a_j + b_j = k_j o_j l_j^2, \quad b_j \geq M_j + \delta_j. \quad (6)$$

The first condition states that the sampled part and the solved part together cover all flattened oil coordinates of local zone j . The second condition ensures that the solved part contains enough coordinates to cover the M_j local equations and leaves δ_j degrees of signing freedom.

At the protocol level, the public parameter set is

$$\text{params} = (q, d, \{k_j\}_{j=1}^{d+1}, \{(v_j, o_j, m_j, l_j, a_j, b_j, \delta_j)\}_{j=1}^{d+1}, \kappa_{\text{sk}}, \kappa_{\text{pk}}, \lambda_{\text{salt}}, H). \quad (7)$$

Here κ_{sk} denotes the bit length of the master secret expansion seed, κ_{pk} denotes the bit length of the public expansion seed derived from the master secret expansion seed, and λ_{salt} denotes the bit length of the signing salt. Thus, (N, M) and the seed lengths determine the public scale of the expanded public MQ map, while the local zone parameters and the margins (a_j, b_j, δ_j) determine the local signing freedom.

2.3 Preliminaries and Notation

If X is a finite set, we write $x \stackrel{\$}{\leftarrow} X$ to denote that x is sampled uniformly at random from X . If A is an algorithm, we write $x \leftarrow A(y)$ to denote that x is the output of A on input y . For a positive integer r , we write $[r] = \{1, \dots, r\}$. The public signature space and the public target space are denoted by $\mathbb{F}_q^N$ and $\mathbb{F}_q^M$, respectively.

For the local zones used in Origami, we write

$$Y = (Y^{(1)}, \dots, Y^{(d+1)}), \quad Y^{(j)} = (Y_1^{(j)}, \dots, Y_{k_j}^{(j)}), \quad (8)$$

where

$$Y_i^{(j)} = (V_i^{(j)}, O_i^{(j)}) \in K_{j,i}^{v_j} \times K_{j,i}^{o_j}. \quad (9)$$

Here $V_i^{(j)}$ and $O_i^{(j)}$ denote the vinegar part and the oil part of the i -th local copy in local zone j , respectively. We also write

$$Y^{(<j)} = (Y^{(1)}, \dots, Y^{(j-1)}) \quad (10)$$

for the prefix of local zones that have already been fixed before local zone j is solved.

The internal central map of Origami is written as

$$F = (F^{(1)}, F^{(2)}, \dots, F^{(d+1)}). \quad (11)$$

For each local zone $j \in [d+1]$, the private oil-coordinate partition maps the flattened oil-coordinate vector Z_j to

$$\rho_j(Z_j) = (U_j, W_j), \quad U_j \in \mathbb{F}_q^{a_j}, \quad W_j \in \mathbb{F}_q^{b_j}. \quad (12)$$

Before solving local zone j , the signer has already fixed the prefix $Y^{(<j)}$, the current-zone vinegar variables, and the sampled part U_j . We write the known variables as

$$X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j). \quad (13)$$

The j -th local zone map is required to be affine in the solved part W_j :

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j, \quad (14)$$

where

$$D_j(X_j) \in \mathbb{F}_q^{M_j}, \quad A_j(X_j) \in \mathbb{F}_q^{M_j \times b_j}. \quad (15)$$

Here D_j is quadratic in the known variables, and A_j is affine in the known variables. No quadratic term in W_j is allowed. Therefore, for a target block $u^{(j)} \in \mathbb{F}_q^{M_j}$, signing in local zone j reduces to the linear system

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j). \quad (16)$$

This affine form in W_j is what allows signing to proceed by sequential linear solving from one local zone to the next.

Equivalently, the defining restriction is that the solved part W_j may appear only affinely in the j -th local zone map, and that the j -th local zone map may depend only

on the prefix variables, the current-zone vinegar variables, and the sampled part U_j . In particular, quadratic terms in W_j are forbidden, and no term may depend on future-zone variables. Subject to these restrictions, the internal central map supports sequential linear solving while keeping the hidden local representations and the private oil-coordinate partitions inside the secret key.

We emphasize that $K_{j,i}$ denotes the actual hidden local algebra used by the i -th local copy in local zone j , rather than merely a single conjugated generator matrix. Concretely, once $S_{j,i}$ and $C_{j,i}$ have been sampled, we define

$$T_{j,i} = C_{j,i}^{-1} S_{j,i} C_{j,i} \quad (17)$$

and then set

$$K_{j,i} = \mathbb{F}_q[T_{j,i}]. \quad (18)$$

Thus, even over the same base field $\mathbb{F}_q$, different local copies may operate in different hidden local algebras.

To simplify the specification of key generation and signing, we use two auxiliary procedures. The first auxiliary procedure reconstructs the hidden local algebra $K_{j,i}$ for the i -th local copy in local zone j directly from the master secret expansion seed seed_{sk} . The matrix-sampling domains are separated by the labels **embed-S** and **embed-C**. Here L_S and L_C denote the numbers of bytes required by the matrix-sampling routine for generating one candidate matrix in $\text{Mat}_{l_j}(\mathbb{F}_q)$, and r_S, r_C are retry counters used only for domain separation between repeated trials. The second auxiliary procedure solves the j -th local zone after the prefix $Y^{(<j)}$, the current-zone vinegar variables, and the sampled part U_j have been fixed.

Algorithm 1 SampleHiddenLocalAlgebra($\text{seed}_{\text{sk}}, j, i, l_j$)

Require: Master secret expansion seed seed_{sk} , local zone index j , local copy index i , matrix dimension l_j

Ensure: Matrix $S_{j,i}$, conjugating matrix $C_{j,i}$, generator $T_{j,i}$, hidden local algebra $K_{j,i}$

```

1:  $r_S \leftarrow 0$ 
2: repeat
3:    $\text{buf}_S \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{embed-S} \parallel j \parallel i \parallel r_S, L_S)$ 
4:    $S_{j,i} \leftarrow \text{PseudorandomMatrix}(\text{buf}_S, l_j, l_j, \mathbb{F}_q)$ 
5:    $r_S \leftarrow r_S + 1$ 
6: until CharacteristicPolynomial( $S_{j,i}$ ) is irreducible over  $\mathbb{F}_q$ 
7:  $r_C \leftarrow 0$ 
8: repeat
9:    $\text{buf}_C \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{embed-C} \parallel j \parallel i \parallel r_C, L_C)$ 
10:   $C_{j,i} \leftarrow \text{PseudorandomMatrix}(\text{buf}_C, l_j, l_j, \mathbb{F}_q)$ 
11:   $r_C \leftarrow r_C + 1$ 
12: until  $\det(C_{j,i}) \neq 0$ 
13:  $T_{j,i} \leftarrow C_{j,i}^{-1} S_{j,i} C_{j,i}$ 
14:  $K_{j,i} \leftarrow \mathbb{F}_q[T_{j,i}]$ 
15: return ( $S_{j,i}, C_{j,i}, T_{j,i}, K_{j,i}$ )
```

Once the known variable tuple

$$X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j) \quad (19)$$

has been fixed, the j -th local zone reduces to a linear system in the solved part W_j :

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j). \quad (20)$$

The following auxiliary procedures formalize hidden local algebra reconstruction and local-zone sequential linear solving.

Algorithm 2 SolveLocalZone($D_j, A_j, X_j, u^{(j)}$)

Require: Local-zone descriptions D_j and A_j

Require: Known variable tuple $X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$

Require: Target block $u^{(j)} \in \mathbb{F}_q^{M_j}$

Ensure: Solved part $W_j \in \mathbb{F}_q^{b_j}$ or $\perp$

```

1:  $d_j \leftarrow \text{Evaluate}(D_j, X_j)$ 
2:  $B_j \leftarrow \text{Evaluate}(A_j, X_j)$ 
3:  $\text{rhs} \leftarrow u^{(j)} - d_j$ 
4: if  $\text{rank}(B_j) \neq M_j$  then
5:   return  $\perp$ 
6: else
7:   Find one solution  $W_j^0$  of  $B_j W_j = \text{rhs}$ 
8:   Sample  $z_j \xleftarrow{\$} \ker(B_j)$ 
9:    $W_j \leftarrow W_j^0 + z_j$ 
10:  return  $W_j$ 
11: end if
```

After flattening the internal signable map into coordinates over the base field $\mathbb{F}_q$, we obtain a multivariate quadratic map

$$G : \mathbb{F}_q^N \longrightarrow \mathbb{F}_q^M. \quad (21)$$

Origami does not publish this expanded map as a full dense coefficient list. Instead, the compressed public key stores a compressed public description

$$pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}}), \quad (22)$$

where seed_{pk} is the public expansion seed and R_{pk} is the explicit residual coefficient vector.

Let $\mathcal{I}_{\text{all}}$ denote the ordered coefficient-position universe of the expanded public MQ map $P_{\text{pub}} : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^M$ under the fixed public-coordinate convention. A coefficient position records the output-equation index and the ordered linear or quadratic input-coordinate indices. The explicit residual coefficients are not selected from all of $\mathcal{I}_{\text{all}}$. Instead, let $\mathcal{I}_{\text{elig}} \subseteq \mathcal{I}_{\text{all}}$ denote the public residual-eligible affine coefficient-position universe. This universe is determined by the parameter set, the local-zone ordering, the output-equation ordering, the prefix or current known-coordinate positions, and the current-zone oil-coordinate positions under the fixed public coefficient-position ordering. Equivalently, these positions correspond to public affine interaction terms between output equations, prefix or current known-coordinate positions, and current-zone oil-coordinate positions. This public eligible-position universe is only a coefficient-scheduling object; it is not a

public description of the private sampled/solved split or of the private oil-coordinate partitions.

The residual-position set is public and deterministic. It is computed as

$$\mathcal{I}_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq \mathcal{I}_{\text{elig}}. \quad (23)$$

All coefficient positions outside $\mathcal{I}_{\text{res}}$ are seed-defined. Thus we write $\mathcal{I}_{\text{seed}} = \mathcal{I}_{\text{all}} \setminus \mathcal{I}_{\text{res}}$.

The public expansion seed generates the coefficients on $\mathcal{I}_{\text{seed}}$, while R_{pk} stores the explicit coefficients on $\mathcal{I}_{\text{res}}$ according to the public rank-based residual-position schedule. More precisely, for each selected eligible coefficient position with canonical linear index idx , the coefficient placed at that public position is $R_{\text{pk}}[\text{rank}(\text{idx})]$. Thus, R_{pk} is not interpreted by a separate private ordering of residual positions; it is interpreted only through the public deterministic map $\text{rank}(\text{idx})$ defined by SelectRes .

The expanded public MQ map is reconstructed as $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{\mathcal{I}_{\text{res}}}(R_{\text{pk}})$, where P_{seed} denotes the seed-defined part on $\mathcal{I}_{\text{seed}}$, and $\text{Embed}_{\mathcal{I}_{\text{res}}}(R_{\text{pk}})$ denotes the sparse MQ map whose coefficients are zero outside $\mathcal{I}_{\text{res}}$ and whose coefficients on selected residual positions are assigned by $R_{\text{pk}}[\text{rank}(\text{idx})]$. Thus, verification uses the expanded public MQ map P_{pub} over the base field $\mathbb{F}_q$, while the hidden local representations, hidden local algebras, private oil-coordinate partitions, and sampled/solved split remain confined to the secret key.

2.4 Hashing and randomness expansion

Origami uses hashing and deterministic randomness expansion for four related purposes: deriving secret local data from the master secret expansion seed, deriving the public expansion seed by domain separation, mapping message-salt pairs to public target vectors, and generating public coefficient data for the compressed public key. The residual-position set used by the compressed public key is public and deterministic, and is derived from the public parameters and the derived public expansion seed. All these uses are separated by explicit context strings, so that secret local data, public coefficient generation, residual-position selection, and hash-to-field outputs are computed in distinct domains.

2.4.1 Hash Function

Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^*$ denote an extendable-output hash function. In practice, H may be instantiated by SHAKE256. All uses of H in Origami are domain-separated by explicit context strings. When a fixed output length of λ bits is required, we write

$$H_{\lambda}(\text{ctx}, X) = H(\text{ctx} \parallel \text{enc}(\lambda) \parallel X)[1..\lambda], \quad (24)$$

where ctx is a domain-separation string, $\text{enc}(\lambda)$ is a fixed-length encoding of the output length, and $[1..\lambda]$ denotes the first λ output bits. This notation is used only to specify length-delimited outputs; the concrete uses for public coefficient generation, secret local data reconstruction, residual-index selection, and hash-to-field sampling are fixed by their own context strings.

2.4.2 Randomness Expansion

We write $\text{Expand}(\text{seed}, \text{ctx}, L) \in \{0, 1\}^{8L}$ for a deterministic expansion procedure that maps a seed, a domain-separation context string ctx , and an output length L in bytes to L pseudorandom output bytes. Concretely, we define

$$\text{Expand}(\text{seed}, \text{ctx}, L) := H(\text{seed} \parallel \text{ctx} \parallel \text{enc}(L))[1..8L], \quad (25)$$

where $\text{enc}(L)$ is a fixed-length encoding of L , and $[1..8L]$ denotes the first $8L$ output bits of the extendable-output hash function.

The context string ctx must be unique and unambiguous for each expansion domain. In particular, different context strings are used for reconstructing hidden local representations and hidden local algebras from the master secret expansion seed, deriving the public expansion seed, generating the seed-defined part of the expanded public MQ map, selecting the public residual-position set, and deriving hash-to-field sampling randomness.

The public expansion seed is derived deterministically from the master secret expansion seed by a dedicated domain-separated call:

$$\text{seed}_{\text{pk}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8). \quad (26)$$

Here param_id is the public identifier of the selected Origami parameter-set instance, and λ_{sec} is the security-level tag of that instance.

The residual-position set is public and deterministic. It is selected from the public residual-eligible affine coefficient-position universe $\mathcal{I}_{\text{elig}} \subseteq \mathcal{I}_{\text{all}}$, which is determined by the public parameters, the local-zone ordering, the output-equation ordering, the prefix or current known-coordinate positions, and the current-zone oil-coordinate positions under the fixed public coefficient-position ordering. More precisely, $\mathcal{I}_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq \mathcal{I}_{\text{elig}}$. The concrete parameter set fixes the number $n_{\text{res}} = |\mathcal{I}_{\text{res}}|$.

To define SelectRes , first order $\mathcal{I}_{\text{elig}}$ lexicographically by local-zone index, output-equation index, first input-coordinate index, and second input-coordinate index. Let $T_{\text{elig}} = |\mathcal{I}_{\text{elig}}|$. The implementation computes a public stride

$$s = \text{Stride}(\text{param_id}, T_{\text{elig}})$$

satisfying

$$\gcd(s, T_{\text{elig}}) = 1,$$

and a public offset

$$o = \text{Offset}(\text{seed}_{\text{pk}}, \text{param_id}, T_{\text{elig}}).$$

For an eligible coefficient position with canonical linear index $\text{idx} \in \{0, \dots, T_{\text{elig}} - 1\}$, define

$$\text{rank}(\text{idx}) = (\text{idx} \cdot s + o) \bmod T_{\text{elig}}.$$

Then

$$\text{idx} \in \mathcal{I}_{\text{res}} \iff \text{rank}(\text{idx}) < |\mathcal{I}_{\text{res}}|.$$

For each selected eligible coefficient position with canonical linear index idx , the corresponding residual coefficient is stored at position $\text{rank}(\text{idx})$ of R_{pk} . Thus, only entries satisfying $\text{rank}(\text{idx}) < |\mathcal{I}_{\text{res}}|$ are used, and the coefficient placed at this public coefficient position is $R_{\text{pk}}[\text{rank}(\text{idx})]$. This residual-position schedule is public and deterministic. It depends only on the public parameter identifier, the public expansion seed, and the fixed eligible-position ordering. It does not use secret information and does not reveal the private sampled/solved split or the private oil-coordinate partitions.

2.4.3 Hashing to Field Vectors

The operation $\text{HashToFieldVector}(X, r, \mathbb{F}_q)$ takes an input string X , expands it by a domain-separated call to the extendable-output hash function H , parses the resulting bit string into field elements of $\mathbb{F}_q$, and outputs a vector in $\mathbb{F}_q^r$. In Origami, the input string X always contains an explicit context tag.

Before target derivation, the message is first compressed to a fixed-length message digest:

$$h_\mu = H_{\text{msg}}(\mu) \in \mathcal{B}^{64}. \quad (27)$$

In the submitted implementation, this digest is defined by the domain-separated 64-byte output

$$H_{\text{msg}}(\mu) = \text{Expand}(\varepsilon, \text{msg} \parallel \text{I2OSP}(|\mu|, 8) \parallel \mu, 64), \quad (28)$$

where ε denotes the empty seed input to the XOF interface. Equivalently, H_{msg} is a fixed 64-byte message-digest procedure shared by signing and verification.

The public target vector used in signing and verification is then derived as

$$t = \text{HashToFieldVector}(\text{target} \parallel h_\mu \parallel \text{salt}, M, \mathbb{F}_q). \quad (29)$$

We also use the notation $\text{ExpandToField}(\text{seed}, \text{ctx}, r)$ for the deterministic procedure that first computes many bytes by $\text{Expand}(\text{seed}, \text{ctx}, L)$, then parses the bit stream into r field elements using the same field-element representation as HashToFieldVector . In the submitted $q = 16$ instantiation, the bit stream is parsed into consecutive four-bit nibbles, and each nibble is interpreted as one element of $\mathbb{F}_{16}$ under the fixed polynomial-basis representation.

If $q = 2^m$, the output stream of H is parsed into consecutive chunks of m bits, and each chunk is interpreted under the fixed representation of $\mathbb{F}_{2^m}$ specified by the parameter set. For a general prime power q , the parameter set must specify a deterministic rejection-sampling rule or another standard encoding procedure that yields a distribution statistically close to uniform over $\mathbb{F}_q^r$. In all cases, the parsing rule, the rejection-sampling convention if one is used, and the canonical representation of field elements must be fixed by the concrete parameter set.

Equivalently, for each fixed field $\mathbb{F}_q$ and dimension r , one may view

$$\text{HashToFieldVector}_{q,r} : \{0, 1\}^* \longrightarrow \mathbb{F}_q^r \quad (30)$$

as a fully specified deterministic map determined by the parameter set and its domain-separation convention.

2.5 Data types and conversions

The Origami protocol manipulates three levels of representation: byte strings, vectors over the base field $\mathbb{F}_q$, and secret local objects used to reconstruct the hidden local representations and hidden local algebras. In addition, the public interface uses a compressed public key consisting of public parameters, a public expansion seed, and a residual coefficient vector. This subsection fixes the corresponding data types, the encoding conventions, the flattening convention for secret local coordinates, and the expansion convention for reconstructing the expanded public MQ map.

2.5.1 Byte Strings

All external interfaces use byte strings. Let $\mathcal{B} = \{0, 1, \dots, 255\}$ denote the set of bytes, let $\mathcal{B}^r$ denote the set of byte strings of length r , and let $\mathcal{B}^*$ denote the set of byte strings of arbitrary length. Concatenation is written $\parallel$. When integers are injected into hashed, expanded, or serialized strings, we write $\text{I2OSP}(j, w)$ for the fixed-width w -byte encoding of a nonnegative integer j .

In Origami, byte strings are used to encode messages, salts, public parameters, the master secret expansion seed, the derived public expansion seed, the residual coefficient vector, and signatures. They are also used in the length encodings and context strings of domain-separated calls to H and Expand .

2.5.2 Base-Field Vectors

The public signature space is $\mathbb{F}_q^N$, while the public target space is $\mathbb{F}_q^M$. The public target vector is split according to the local-zone output dimensions as

$$t = (u^{(1)}, \dots, u^{(d+1)}), \quad u^{(j)} \in \mathbb{F}_q^{M_j}. \quad (31)$$

For each local zone j , the dimension M_j is given by

$$M_j = m_j l_j^2. \quad (32)$$

The flattened internal assignment of the whole signature lies in $\mathbb{F}_q^N$, where

$$N = \sum_{j=1}^{d+1} k_j (v_j + o_j) l_j^2. \quad (33)$$

2.5.3 Hidden Local Representations

Before flattening, the secret computation is carried out in hidden local representations defined over the hidden local algebras

$$K_{j,i} \subseteq \text{Mat}_{l_j}(\mathbb{F}_q), \quad j \in \{1, \dots, d+1\}, \quad i \in \{1, \dots, k_j\}. \quad (34)$$

Accordingly, each local internal variable of the i -th local copy in local zone j is represented as an element of $K_{j,i}$, and each such element occupies l_j^2 coordinates once expanded over the base field $\mathbb{F}_q$.

The global flattening map is written as

$$\text{Flat} : \prod_{j=1}^{d+1} \prod_{i=1}^{k_j} K_{j,i}^{v_j + o_j} \longrightarrow \mathbb{F}_q^N. \quad (35)$$

When the hidden local representations and private oil-coordinate partitions are known, the corresponding inverse conversion is written abstractly as Unflat . This inverse conversion is used only by the signer when reconstructing secret local coordinates; it is not part of the compressed public key or the verification interface.

2.5.4 Generic Vector Encoding

For each integer $r \geq 1$, we fix an injective encoding

$$\text{EncodeVec}_q : \mathbb{F}_q^r \longrightarrow \mathcal{B}^{L_q(r)} \quad (36)$$

together with its inverse on valid encodings,

$$\text{DecodeVec}_q : \mathcal{B}^{L_q(r)} \longrightarrow \mathbb{F}_q^r. \quad (37)$$

If $q = 2^m$, each field element is represented by m bits, and the resulting bit string is packed into bytes. In this case, the encoded length is

$$L_q(r) = \left\lceil \frac{rm}{8} \right\rceil. \quad (38)$$

The concrete parameter set must fix a canonical representation of $\mathbb{F}_{2^m}$ by specifying an irreducible polynomial, so that both field arithmetic and field-element encoding are uniquely determined. In the submitted $q = 16$ instantiation, the field $\mathbb{F}_{16}$ is represented as

$$\mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4 + x + 1).$$

Each field element is represented by the corresponding four-bit polynomial-basis coefficient vector. Under this convention, each field element occupies one nibble, and a vector in $\mathbb{F}_{16}^r$ is encoded into $\left\lceil \frac{r}{2} \right\rceil$ bytes.

2.5.5 Encoding Hidden Local Objects and Public Coefficients

Whenever a hidden local algebra object or a local variable tuple is serialized, it is first expanded into its coordinate representation over the base field $\mathbb{F}_q$, then written in a globally fixed order, and finally encoded via EncodeVec_q . This ordering must be fixed once and for all in order to make serialization deterministic.

Concretely, matrices in $\text{Mat}_{l_j}(\mathbb{F}_q)$ are written in row-major order. Tuples of local variables are written first by local zone index j , then by local copy index i , and then by vinegar/oil order within each local copy. Thus, for a local zone j , the ordering of the secret local coordinates follows

$$Y^{(j)} = (V_1^{(j)}, O_1^{(j)}, \dots, V_{k_j}^{(j)}, O_{k_j}^{(j)}). \quad (39)$$

For public coefficient scheduling, Origami fixes an ordering on the eligible affine coefficient positions, rather than on the full dense MQ coefficient universe. We write this public eligible position set as $\mathcal{I}_{\text{elig}}$. It is determined by the parameter set, the local-zone ordering, the output-equation ordering, the prefix or current known-coordinate positions, and the current-zone oil-coordinate positions under the fixed public coefficient-position ordering. Equivalently, these positions correspond to public affine interaction terms between output equations, prefix or current known-coordinate positions, and current-zone oil-coordinate positions. This public eligible-position universe is only a coefficient-scheduling object; it is not a public description of the private sampled/solved split or of the private oil-coordinate partitions.

For residual coefficient scheduling, Origami fixes a public deterministic rank-based selection rule on the eligible affine coefficient positions, rather than selecting residual

coefficients from the full dense MQ coefficient universe. We write this public residual-eligible position set as $\mathcal{I}_{\text{elig}} \subseteq \mathcal{I}_{\text{all}}$.

The residual-position set is $\mathcal{I}_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq \mathcal{I}_{\text{elig}}$, and the seed-generated position set is $\mathcal{I}_{\text{seed}} = \mathcal{I}_{\text{all}} \setminus \mathcal{I}_{\text{res}}$. The public expansion seed generates the coefficients on $\mathcal{I}_{\text{seed}}$, while the residual coefficient vector R_{pk} stores the explicit coefficients on $\mathcal{I}_{\text{res}}$ according to the public rank-based residual-position schedule. More precisely, for each selected eligible coefficient position with canonical linear index idx , the coefficient placed at that public position is $R_{\text{pk}}[\text{rank}(\text{idx})]$.

The set $\mathcal{I}_{\text{res}}$ is distributed across public eligible coefficient positions associated with local zones, output equations, prefix or current known-coordinate positions, and current-zone oil-coordinate interaction terms. It is not a public encoding of the private sampled/solved split, and it does not reveal the private oil-coordinate partitions. In the $q = 16$ instantiation, R_{pk} is the vector of $\mathbb{F}_{16}$ coefficients interpreted through the public rank-based residual-position schedule.

Therefore, the compressed public key does not serialize the full dense coefficient array of the expanded public MQ map. It serializes the public expansion seed and the residual coefficient vector needed for the eligible affine coefficient positions, together with the public parameter identifier.

2.5.6 Flattening Convention

To make the definition of the expanded public MQ map completely unambiguous, we fix a local-zone-by-local-zone flattening rule. For each local zone j , each local copy i , and each hidden local algebra element $X \in K_{j,i} \subseteq \text{Mat}_{l_j}(\mathbb{F}_q)$, we define

$$\text{Flat}_{j,i}(X) \in \mathbb{F}_q^{l_j^2} \quad (40)$$

by expanding X into its matrix coordinates over the base field $\mathbb{F}_q$ and listing these coordinates in row-major order.

This convention extends componentwise to tuples of local variables. Within each local copy, the vinegar part precedes the oil part. Within each local zone, local copies are concatenated in the order $i = 1, \dots, k_j$. The local zones are then concatenated in the order $j = 1, \dots, d + 1$. Thus,

$$\text{Flat}(Y) = \text{Flat}(Y^{(1)}, \dots, Y^{(d+1)}) \in \mathbb{F}_q^N. \quad (41)$$

The same ordering convention is used for local output blocks:

$$\text{FlatOut}(F^{(1)}(Y^{(\leq 1)}), \dots, F^{(d+1)}(Y^{(\leq d+1)})) \in \mathbb{F}_q^M. \quad (42)$$

The flattened internal signable map G is therefore the unique multivariate quadratic map over $\mathbb{F}_q$ such that

$$G(\text{Flat}(Y)) = \text{FlatOut}(F(Y)) \quad (43)$$

for every internal assignment Y .

The public signature coordinates use a public variable permutation derived from the public expansion seed. We write $\Pi_{\text{pub}} = \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$ for this permutation on $\mathbb{F}_q^N$. It is public because it is derived from the public expansion seed. The

parameter identifier is already bound to seed_{pk} through the seed-derivation rule, and the value N determines the length of the generated permutation.

The procedure $\text{Permute}(\text{seed}, \text{ctx}, r)$ is deterministic. It initializes the ordered list $L = (1, 2, \dots, r)$ and uses domain-separated output bytes $\text{Expand}(\text{seed}, \text{ctx} \parallel \text{I2OSP}(r, 4) \parallel c, L_c)$ to perform an XOF-based Fisher–Yates shuffle of L . At step $s \in \{r, r-1, \dots, 2\}$, the procedure samples an integer $a_s \in \{1, \dots, s\}$ by rejection sampling from the expanded byte stream and swaps L_s and L_{a_s} . The resulting list L defines the permutation. The same deterministic procedure is used for all calls to Permute , with different domain-separation strings.

Thus, if

$$y = \text{Flat}(Y) \in \mathbb{F}_q^N \quad (44)$$

is the flattened internal assignment, then the public signature coordinate vector is represented in the permuted coordinate order

$$\sigma = \Pi_{\text{pub}}(y). \quad (45)$$

Equivalently, the expanded public MQ map reconstructed from the compressed public key satisfies

$$P_{\text{pub}}(\sigma) = G(\Pi_{\text{pub}}^{-1}(\sigma)). \quad (46)$$

The expanded public MQ map P_{pub} therefore has the same input and output vector spaces as G , but its input coordinates follow the public coordinate order determined by Π_{pub} .

2.5.7 Flattening and Signature Encoding

After the sequential linear solving terminates successfully, the signer obtains an internal assignment

$$Y = (Y^{(1)}, \dots, Y^{(d+1)}). \quad (47)$$

This assignment is first converted into the flattened internal vector $y = \text{Flat}(Y) \in \mathbb{F}_q^N$.

The public signature vector is then obtained by applying the public variable permutation derived from the public expansion seed $\sigma = \Pi_{\text{pub}}(y) \in \mathbb{F}_q^N$. The public signature vector is serialized as $\text{EncodeVec}_q(\sigma)$.

Abstractly, a signature consists of the pair (σ, salt) used in the signing and verification algorithms, and its wire encoding is the byte string $\text{sig} = \text{EncodeVec}_q(\sigma) \parallel \text{salt}$. When $q = 16$, the encoded length of σ is $\lceil \frac{N}{2} \rceil$ bytes, and the total signature length is $\lceil \frac{N}{2} \rceil + \frac{\lambda_{\text{salt}}}{8}$ bytes.

2.5.8 Public-Key and Secret-Key Serialization

In this section, the public key is not identified with a full dense coefficient list of the public MQ map. Instead, Origami uses the compressed public key

$$pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}}), \quad (48)$$

where params is the public parameter description, seed_{pk} is the public expansion seed published in the public key, and R_{pk} is the explicit residual coefficient vector.

The public expansion seed is derived deterministically from the master secret expansion seed during key generation:

$$\text{seed}_{\text{pk}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8). \quad (49)$$

Here param_id is the public identifier of the selected Origami parameter-set instance, and λ_{sec} is the security-level tag of that instance. The derived public expansion seed is then published as part of pk . It is not stored as a separate component of sk , and it is not treated as an independent secret in the security analysis.

The abstract compressed public key is written as

$$pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}}). \quad (50)$$

At the byte-string interface, the submitted implementation encodes it as

$$\text{EncodePK}(pk) = \text{LE32}(\text{param_id}) \parallel \text{seed}_{\text{pk}} \parallel \text{EncodeVec}_{16}(R_{\text{pk}}). \quad (51)$$

Thus, the public parameters are represented by the four-byte parameter identifier in the serialized public key, while the full parameter set is fixed by the selected Origami instance.

During verification, the verifier expands the compressed public key to reconstruct the expanded public MQ map. The ordered public coefficient-position universe I_{all} and the residual-eligible affine coefficient-position universe $I_{\text{elig}} \subseteq I_{\text{all}}$ are first determined from the selected parameter instance. The residual-position set is then recomputed as $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$, and the seed-generated position set is $I_{\text{seed}} = I_{\text{all}} \setminus I_{\text{res}}$. The verifier then reconstructs $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$, $P_{\text{pub}} : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^M$. Thus, the compressed public key does not store all coefficients of P_{pub} explicitly. It stores the public expansion seed and the residual coefficient vector needed for the eligible affine coefficient positions. The residual coefficient vector R_{pk} is interpreted through the public rank-based residual-position schedule. More precisely, for each selected eligible coefficient position with canonical linear index idx , the coefficient placed at that public position is $R_{\text{pk}}[\text{rank}(\text{idx})]$. Thus, the verifier does not use a private residual ordering; it reconstructs the expanded public MQ map using only the public rule defined by SelectRes .

The compact secret key is only the master secret expansion seed:

$$sk = \text{seed}_{\text{sk}}. \quad (52)$$

The hidden local representations, hidden local algebras, private oil-coordinate partitions, sampled/solved split, and signable internal representation are reconstructed deterministically from seed_{sk} and the public parameters. They are not serialized as separate secret-key components.

2.5.9 Encoding Requirement

All encoding and decoding procedures must satisfy the expected inverse property on valid encodings. In particular, for every $x \in \mathbb{F}_q^r$, one must have

$$\text{DecodeVec}_q(\text{EncodeVec}_q(x)) = x. \quad (53)$$

The same requirement applies to byte strings, field vectors, hidden local algebra objects, flattened assignments, signatures, and residual coefficient vectors.

In addition, public-key expansion must be deterministic. For every compressed public key $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$, the same public expansion seed, residual coefficient vector, eligible coefficient-position ordering, public variable permutation, and deterministic residual-selection rule must reconstruct the same expanded public MQ map.

More precisely, the ordered public coefficient-position universe I_{all} and the residual-eligible coefficient-position universe $I_{\text{elig}} \subseteq I_{\text{all}}$ are determined by the public parameters and the fixed zone and coordinate ordering. The residual-position set is then computed as $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$, and the seed-generated positions are $I_{\text{seed}} = I_{\text{all}} \setminus I_{\text{res}}$. The public variable permutation is also reconstructed deterministically as $\Pi_{\text{pub}} = \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$. Here param_id has already been absorbed into seed_{pk} during public-seed derivation, and N specifies the permutation length. With these conventions fixed, the verifier reconstructs $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$, where the expansion is interpreted in the public coordinate order determined by Π_{pub} . Thus, verification is well-defined: the verifier obtains the same expanded public MQ map from the compressed public key whenever the public parameters, encoding rules, residual-selection rule, and public permutation rule are fixed.

2.6 The Basic Origami functionalities

We define three basic functionalities that constitute the Origami signature scheme:

- **Origami.KeyGen:** The key generation algorithm that produces a compressed public key and a compact secret key.
- **Origami.Sign:** The signature generation algorithm that takes the compact secret key and a message, reconstructs the hidden local representations, hidden local algebras, and private oil-coordinate partitions from the master secret expansion seed, and then produces a signature by sequential linear solving.
- **Origami.Verify:** The verification algorithm that expands the compressed public key into the expanded public MQ map and checks the validity of a signature by ordinary MQ evaluation.

The three functionalities are specified in the order in which they are used in the signature scheme. We first describe the key generation procedure, which constructs the secret local data, determines the residual coefficient vector, and outputs the compressed public key. We then describe the signing procedure, which reconstructs the signable internal representation and solves the local zones sequentially. Finally, we describe the verification procedure, which reconstructs the expanded public MQ map from the compressed public key and checks the MQ evaluation equation.

2.6.1 Key Generation Algorithm

Origami.KeyGen takes as input the public parameter set

$$\text{params} = (q, d, \{k_j\}_{j=1}^{d+1}, \{(v_j, o_j, m_j, l_j, a_j, b_j, \delta_j)\}_{j=1}^{d+1}, \kappa_{\text{sk}}, \kappa_{\text{pk}}, \lambda_{\text{salt}}, H)$$

and outputs a compressed public key pk and a compact secret key sk .

Algorithm 3 Origami.KeyGen(params)

Require: Public parameter set params with identifier param_id

Ensure: Compressed public key pk , compact secret key sk

```
1: Sample  $\text{seed}_{\text{sk}} \xleftarrow{\$} \{0, 1\}^{\kappa_{\text{sk}}}$ 
2: Derive
    $\text{seed}_{\text{pk}} \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8)$ 
3: Derive
    $\text{seed}_{\text{coeff}} \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{coeff-seed} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{sk}}/8, \kappa_{\text{sk}}/8)$ 
4: for  $j = 1$  to  $d + 1$  do
5:   for  $i = 1$  to  $k_j$  do
6:      $(S_{j,i}, C_{j,i}, T_{j,i}, K_{j,i}) \leftarrow \text{SampleHiddenLocalAlgebra}(\text{seed}_{\text{sk}}, j, i, l_j)$ 
7:   end for
8:   Generate the private oil-coordinate partition  $\rho_j : Z_j \mapsto (U_j, W_j)$ 
9: end for
10: for  $j = 1$  to  $d + 1$  do
11:   Generate the local maps  $D_j$  and  $A_j$  such that
      $F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j$ 
12:   Ensure that no quadratic term in  $W_j$  appears
13: end for
14: Determine the ordered public coefficient-position universe  $I_{\text{all}}$ 
15: Determine the public residual-eligible affine coefficient-position universe  $I_{\text{elig}} \subseteq I_{\text{all}}$ 
16:  $I_{\text{res}} \leftarrow \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}})$ 
17:  $I_{\text{seed}} \leftarrow I_{\text{all}} \setminus I_{\text{res}}$ 
18:  $R_{\text{pk}} \leftarrow \text{ExpandToField}(\text{seed}_{\text{coeff}}, \text{Rpk-coeff} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel |I_{\text{res}}|, |I_{\text{res}}|)$ 
19: Generate the seed-defined public coefficient part  $P_{\text{seed}}$  from  $\text{seed}_{\text{pk}}$  on  $I_{\text{seed}}$ 
20:  $P_{\text{pub}} \leftarrow P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$ 
21:  $pk \leftarrow (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$ 
22:  $sk \leftarrow \text{seed}_{\text{sk}}$ 
23: return  $(pk, sk)$ 
```

The algorithm first samples only the master secret expansion seed:

$$\text{seed}_{\text{sk}} \xleftarrow{\$} \{0, 1\}^{\kappa_{\text{sk}}} \quad (54)$$

The public expansion seed is then derived from seed_{sk} by a domain-separated expansion:

$$\text{seed}_{\text{pk}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8). \quad (55)$$

The master secret expansion seed is used to reconstruct the hidden local representations, hidden local algebras, private oil-coordinate partitions, and the signable internal representation. The derived public expansion seed is published as part of the compressed public key and is used to generate the seed-defined part of the expanded public MQ map. Thus, seed_{pk} is not sampled as an independent secret and is not stored as a separate component of the secret key. For each local zone $j \in \{1, \dots, d + 1\}$ and each local copy $i \in \{1, \dots, k_j\}$, the algorithm reconstructs the hidden local algebra $K_{j,i}$ directly from seed_{sk} using the domain-separated labels **embed-S** and **embed-C**. It also generates the private oil-coordinate partition

$$\rho_j : Z_j \mapsto (U_j, W_j), \quad U_j \in \mathbb{F}_q^{a_j}, \quad W_j \in \mathbb{F}_q^{b_j}. \quad (56)$$

For each local zone, the algorithm generates the local maps in the affine form

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j, \quad (57)$$

where $X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$. The generation enforces that no quadratic term in W_j appears, so that signing in local zone j reduces to sequential linear solving.

Finally, the algorithm determines the ordered public coefficient-position universe I_{all} and the public residual-eligible affine coefficient-position universe $I_{\text{elig}} \subseteq I_{\text{all}}$. It then computes the public residual-position set $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$, and sets $I_{\text{seed}} = I_{\text{all}} \setminus I_{\text{res}}$. The public expansion seed generates the coefficients on I_{seed} , while the residual coefficient vector R_{pk} stores the explicit coefficients on I_{res} .

To make the residual coefficients reproducible from the compact secret key, the algorithm also derives a coefficient seed from the master secret expansion seed:

$$\text{seed}_{\text{coeff}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{coeff-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{sk}}/8, \kappa_{\text{sk}}/8). \quad (58)$$

It then generates the residual coefficient vector by

$$R_{\text{pk}} = \text{ExpandToField}(\text{seed}_{\text{coeff}}, \text{Rpk-coeff} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel |\mathcal{I}_{\text{res}}|, |\mathcal{I}_{\text{res}}|). \quad (59)$$

Here $|\mathcal{I}_{\text{res}}|$ is the number of residual $\mathbb{F}_{16}$ coefficients. Since $q = 16$, two residual coefficients are packed into one byte, and hence $|R_{\text{pk}}|_{\text{bytes}} = |\mathcal{I}_{\text{res}}|/2$. Thus, R_{pk} is published in the compressed public key, while the signer can reconstruct the same residual coefficient vector from seed_{sk} without taking pk as an explicit input. The expanded public MQ map satisfies $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$. The coefficient schedule used to reconstruct P_{pub} is the same schedule used to reconstruct the signable internal representation. Therefore, for every internal assignment Y , if $y = \text{Flat}(Y)$, $\sigma = \Pi_{\text{pub}}(y)$, then the expanded public MQ map satisfies the consistency relation $P_{\text{pub}}(\sigma) = G(y) = \text{FlatOut}(F(Y))$. This relation is part of the key-generation invariant and is the only relation between the compressed public key and the secret signable representation needed for correctness.

The output keys are $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$, $sk = \text{seed}_{\text{sk}}$. In this section, we use the compact secret-key convention. The secret key stores only the secret expansion seed $sk = \text{seed}_{\text{sk}}$, together with a parameter-set identifier when the public parameters are not implicit. The expanded secret local data are reconstructed deterministically from seed_{sk} and params . These reconstructed data include the hidden local representations, the hidden local algebras, the private oil-coordinate partitions, the sampled/solved split, and the signable internal representation used by the signing algorithm.

The compressed public key stores $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$. Thus, the key generation algorithm outputs a compact signer-side seed and a compressed public description from which the expanded public MQ map can be reconstructed during verification.

2.6.2 Sign Algorithm

`Origami.Sign` takes as input a compact secret key sk and a message μ , and outputs a signature (σ, salt) . The signer first parses $sk = \text{seed}_{\text{sk}}$. Using seed_{sk} and the fixed public parameters of the selected instance, the signer reconstructs the hidden local representations, hidden local algebras, private oil-coordinate partitions, sampled/solved split, and signable internal representation. In particular, each hidden local algebra is reconstructed

directly from seed_{sk} using the domain-separated labels **embed-S** and **embed-C**. The compressed public key is not an explicit input to the signing algorithm.

Algorithm 4 Origami.Sign(sk, μ)

Require: Compact secret key sk , message $\mu \in \mathcal{B}^*$

Ensure: Signature $\text{sig} = (\sigma, \text{salt})$

```

1: Parse  $sk$  as  $\text{seed}_{\text{sk}}$ 
2: Use the fixed public parameters of the selected Origami instance
3: Derive  $\text{seed}_{\text{pk}} \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8)$ 
4: Derive  $\Pi_{\text{pub}} \leftarrow \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$ 
5: Derive  $\text{seed}_{\text{coeff}} \leftarrow \text{Expand}(\text{seed}_{\text{sk}}, \text{coeff-seed} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{sk}}/8, \kappa_{\text{sk}}/8)$ 
6:  $\mathcal{I}_{\text{res}} \leftarrow \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}})$ 
7:  $R_{\text{coeff}} \leftarrow \text{ExpandToField}(\text{seed}_{\text{coeff}}, \text{Rpk-coeff} \parallel \text{param\_id} \parallel \lambda_{\text{sec}} \parallel |\mathcal{I}_{\text{res}}|, |\mathcal{I}_{\text{res}}|)$ 
8: Reconstruct the hidden local representations, hidden local algebras, private oil-coordinate partitions, sampled/solved split, residual coefficients, and signable internal representation from  $\text{seed}_{\text{sk}}$ ,  $R_{\text{coeff}}$ , and  $\text{params}$ 
9:  $M \leftarrow \sum_{j=1}^{d+1} M_j$ ;  $\text{salt} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$ ;  $h_{\mu} \leftarrow H_{\text{msg}}(\mu)$ 
10:  $t \leftarrow \text{HashToFieldVector}(\text{target} \parallel h_{\mu} \parallel \text{salt}, M, \mathbb{F}_q)$ 
11: Split  $t = (u^{(1)}, \dots, u^{(d+1)})$  with  $u^{(j)} \in \mathbb{F}_q^{M_j}$ 
12:  $Y^{(<1)} \leftarrow \emptyset$ 
13: for  $j = 1$  to  $d + 1$  do
14:   repeat
15:     for  $i = 1$  to  $k_j$  do
16:        $V_i^{(j)} \xleftarrow{\$} K_{j,i}^{v_j}$ 
17:     end for
18:      $U_j \xleftarrow{\$} \mathbb{F}_q^{a_j}$ 
19:      $X_j \leftarrow (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$ 
20:      $W_j \leftarrow \text{SolveLocalZone}(D_j, A_j, X_j, u^{(j)})$ 
21:   until  $W_j \neq \perp$ 
22:   Recover  $Z_j \leftarrow \rho_j^{-1}(U_j, W_j)$ 
23:   Recover  $Y^{(j)}$  from the current-zone vinegar variables and  $Z_j$ 
24:    $Y^{(<j+1)} \leftarrow (Y^{(<j)}, Y^{(j)})$ 
25: end for
26:  $Y \leftarrow (Y^{(1)}, Y^{(2)}, \dots, Y^{(d+1)})$ 
27:  $y \leftarrow \text{Flat}(Y)$ 
28:  $\sigma \leftarrow \Pi_{\text{pub}}(y)$ 
29: return  $(\sigma, \text{salt})$ 

```

Next, the signer derives the same public expansion seed from seed_{sk} by the key-generation domain-separation rule:

$$\text{seed}_{\text{pk}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8). \quad (60)$$

It then recomputes the public residual-position set $\mathcal{I}_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}})$ using the fixed parameter-set instance.

Although R_{pk} is stored as verifier-side public data, the signer reconstructs the same residual coefficient vector from a coefficient seed derived from seed_{sk} . More precisely, the signer derives $\text{seed}_{\text{coeff}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{coeff-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{sk}}/8, \kappa_{\text{sk}}/8)$, and reconstructs $R_{\text{coeff}} = \text{ExpandToField}(\text{seed}_{\text{coeff}}, \text{Rpk-coeff} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel |\mathcal{I}_{\text{res}}|, |\mathcal{I}_{\text{res}}|)$. The reconstructed vector R_{coeff} is consistent with the R_{pk} component published in the compressed public key, because both vectors are derived from the same coefficient seed and the same domain-separated residual-coefficient expansion rule. Thus, signing can construct the same local linear systems without taking pk as an explicit input.

The signer samples a fresh salt $\text{salt} \xleftarrow{\$} \{0, 1\}^{\lambda_{\text{salt}}}$, and first computes the fixed-length message digest $h_\mu = H_{\text{msg}}(\mu) \in \mathcal{B}^{64}$. The public target vector is then derived as $t = \text{HashToFieldVector}(\text{target} \parallel h_\mu \parallel \text{salt}, M, \mathbb{F}_q) \in \mathbb{F}_q^M$. The target vector is then split according to the local-zone output dimensions: $t = (u^{(1)}, \dots, u^{(d+1)})$, $u^{(j)} \in \mathbb{F}_q^{M_j}$.

No public output hiding is undone during signing; the signer works directly with the target blocks of the expanded public MQ map and the reconstructed signable internal representation. The signer then proceeds through the local zones sequentially. For each local zone $j \in \{1, \dots, d+1\}$, the signer samples the current-zone vinegar variables and the sampled part of the flattened oil coordinates:

$$V_i^{(j)} \xleftarrow{\$} K_{j,i}^{v_j} \quad \text{for } i = 1, \dots, k_j, \quad U_j \xleftarrow{\$} \mathbb{F}_q^{a_j}. \quad (61)$$

It then forms the known variable tuple $X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$. and evaluates $D_j(X_j)$ and $A_j(X_j)$. If $\text{rank}(A_j(X_j)) \neq M_j$, the signer resamples the current-zone vinegar variables and U_j . Otherwise, it solves $A_j(X_j)W_j = u^{(j)} - D_j(X_j)$ by finding one solution W_j^0 and sampling a random kernel element $z_j \in \ker(A_j(X_j))$, then setting

$$W_j = W_j^0 + z_j. \quad (62)$$

Finally, the signer recovers the flattened oil-coordinate vector by $Z_j = \rho_j^{-1}(U_j, W_j)$, and hence obtains the current local assignment $Y^{(j)}$. Once all local zones have been solved, the resulting internal assignment is $Y = (Y^{(1)}, \dots, Y^{(d+1)})$. The signer then applies the fixed flattening convention and obtains the internal flattened vector $y = \text{Flat}(Y) \in \mathbb{F}_q^N$. It can be used to compute $\sigma = \Pi_{\text{pub}}(y) \in \mathbb{F}_q^N$. Thus, we have a final signature pair (σ, salt) .

2.6.3 Verify Algorithm

Origami.Verify takes as input a compressed public key pk , a message μ , and a signature (σ, salt) .

Algorithm 5 Origami.Verify(pk, μ, sig)

Require: Compressed public key pk , message $\mu \in \mathcal{B}^*$, signature $\text{sig} = (\sigma, \text{salt})$

Ensure: Integer result (1 for valid, 0 for invalid)

- 1: Parse pk as $(\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$
 - 2: Determine N , M , and the ordered public coefficient-position universe I_{all} from params
 - 3: Determine the public residual-eligible affine coefficient-position universe $I_{\text{elig}} \subseteq I_{\text{all}}$ from params
 - 4: $I_{\text{res}} \leftarrow \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}})$
 - 5: $I_{\text{seed}} \leftarrow I_{\text{all}} \setminus I_{\text{res}}$
 - 6: Generate the seed-defined part P_{seed} from seed_{pk} on I_{seed}
 - 7: Reconstruct the public variable permutation
 $\Pi_{\text{pub}} \leftarrow \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$
 - 8: Reconstruct the expanded public MQ map
 $P_{\text{pub}} \leftarrow P_{\text{seed}} + \text{Embed}_{\mathcal{I}_{\text{res}}}(R_{\text{pk}})$
 with the public coordinate convention induced by Π_{pub}
 - 9: $h_{\mu} \leftarrow H_{\text{msg}}(\mu)$
 - 10: $t \leftarrow \text{HashToFieldVector}(\text{target} \parallel h_{\mu} \parallel \text{salt}, M, \mathbb{F}_q)$
 - 11: $z \leftarrow P_{\text{pub}}(\sigma)$
 - 12: **if** $z = t$ **then**
 - 13: **return** 1
 - 14: **else**
 - 15: **return** 0
 - 16: **end if**
-

The verifier first parses the public key as $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$. From params and seed_{pk} , the verifier recomputes the ordered public coefficient-position universe I_{all} , the residual-eligible affine coefficient-position universe $I_{\text{elig}} \subseteq I_{\text{all}}$, and the public residual-position set $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$. We can set $I_{\text{seed}} = I_{\text{all}} \setminus I_{\text{res}}$. Next, we reconstruct the public variable permutation $\Pi_{\text{pub}} = \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$, where the parameter identifier is already bound to seed_{pk} , and N determines the permutation length. Using the public expansion seed, the residual coefficient vector, the eligible coefficient-position ordering, and the public permutation convention, the verifier reconstructs the expanded public MQ map $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{\mathcal{I}_{\text{res}}}(R_{\text{pk}})$, $P_{\text{pub}} : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^M$. Equivalently, if G denotes the flattened internal signable map, then the public input coordinate convention satisfies $P_{\text{pub}}(\sigma) = G(\Pi_{\text{pub}}^{-1}(\sigma))$. The verifier computes the fixed-length message digest $h_{\mu} = H_{\text{msg}}(\mu) \in \mathcal{B}^{64}$, and derives the public target vector $t = \text{HashToFieldVector}(\text{target} \parallel h_{\mu} \parallel \text{salt}, M, \mathbb{F}_q)$. It accepts if and only if the MQ evaluation equation $P_{\text{pub}}(\sigma) = t$ holds.

2.6.4 Correctness

Theorem 1 (Correctness) *Assume that (pk, sk) is generated by $\text{Origami.KeyGen}(\text{params})$ for the selected Origami parameter-set instance, where $sk = \text{seed}_{\text{sk}}$, and $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$, where the public expansion seed is derived from the master secret expansion seed by the key-generation domain-separation rule $\text{seed}_{\text{pk}} = \text{Expand}(\text{seed}_{\text{sk}}, \text{pk-seed} \parallel \text{param_id} \parallel \lambda_{\text{sec}} \parallel \kappa_{\text{pk}}/8, \kappa_{\text{pk}}/8)$. Let μ be any message. Suppose that one execution of $\text{Origami.Sign}(sk, \mu)$*

terminates successfully, i.e., for every local zone $j \in \{1, \dots, d+1\}$ the signer finds current-zone vinegar variables and a sampled part U_j such that $\text{rank}(A_j(X_j)) = M_j$, and obtains a solved part W_j satisfying $A_j(X_j)W_j = u^{(j)} - D_j(X_j)$. If the algorithm outputs (σ, salt) , then

$$\text{Origami.Verify}(pk, \mu, (\sigma, \text{salt})) = 1. \quad (63)$$

Proof. For verification, we can parse the public key as $pk = (\text{params}, \text{seed}_{\text{pk}}, R_{\text{pk}})$. By the public-key expansion rule, the verifier determines I_{all} and $I_{\text{elig}} \subseteq I_{\text{all}}$ from the selected parameter-set instance and then computes $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$ and $I_{\text{seed}} = I_{\text{all}} \setminus I_{\text{res}}$. Next, the verifier computes $\Pi_{\text{pub}} = \text{Permute}(\text{seed}_{\text{pk}}, \text{secret-to-public}, N)$. The parameter identifier has already been bound to seed_{pk} by the key-generation seed-derivation rule, and N fixes the length of the generated permutation. Hence the verifier reconstructs the expanded public MQ map

$$P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}}), \quad P_{\text{pub}} : \mathbb{F}_q^N \longrightarrow \mathbb{F}_q^M, \quad (64)$$

with the public input-coordinate convention induced by Π_{pub} .

For a message μ , both the signer and the verifier compute the fixed-length digest $h_\mu = H_{\text{msg}}(\mu) \in \mathcal{B}^{64}$. They then compute the same public target vector

$$t = \text{HashToFieldVector}(\text{target} \parallel h_\mu \parallel \text{salt}, M, \mathbb{F}_q). \quad (65)$$

The signing algorithm is defined for the same fixed Origami parameter-set instance. Therefore, the signer does not parse the compressed public key during signing. Instead, it uses the fixed public parameters of the selected instance and reconstructs the secret local data from seed_{sk} and params . The signer splits the target vector as $t = (u^{(1)}, \dots, u^{(d+1)})$, where $u^{(j)} \in \mathbb{F}_q^{M_j}$. No public output hiding is inverted during signing. The target blocks $u^{(j)}$ are used directly in the local-zone equations of the reconstructed signable internal representation.

The signer processes the local zones in order. For each local zone $j \in \{1, \dots, d+1\}$, after the prefix $Y^{(<j)}$, the current-zone vinegar variables, and the sampled part U_j have been fixed, the known variable tuple is $X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$. The j -th local zone map has the affine form

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j. \quad (66)$$

Whenever the signing algorithm exits the repeat-loop for local zone j , the procedure `SolveLocalZone` has returned some $W_j \neq \perp$. By the definition of `SolveLocalZone`, this means that

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j). \quad (67)$$

Substituting this equality into the affine form of $F^{(j)}$ gives $F^{(j)}(Y^{(\leq j)}) = u^{(j)}$.

Since this holds for every local zone $j \in \{1, \dots, d+1\}$, the full internal assignment $Y = (Y^{(1)}, \dots, Y^{(d+1)})$ satisfies

$$F(Y) = (u^{(1)}, \dots, u^{(d+1)}) = t. \quad (68)$$

After flattening $y = \text{Flat}(Y) \in \mathbb{F}_q^N$, we obtain

$$G(y) = \text{FlatOut}(F(Y)) = t. \quad (69)$$

Table 1 – Implementation platform and software configuration.

Item	Value
Implementation language	C
Implementation paths	Portable ISO C reference implementation and general 64-bit optimized implementation
Compiler	gcc 11.4.0
Reference compile flags	<code>-std=c11 -O3 -Wall -Wextra -Wpedantic</code> <code>-Wshadow -Wpointer-arith -DORIGAMI_OPT=REF</code> <code>-DORIGAMI_ENABLE_STATS</code>
Optimized compile flags	<code>-std=c11 -O3 -Wall -Wextra -Wpedantic</code> <code>-Wshadow -Wpointer-arith -DORIGAMI_OPT=OPT</code> <code>-DORIGAMI_OPTIMIZED_PROFILE -DORIGAMI_ENABLE_STATS</code>
CPU	Intel(R) Core(TM) i5-10210U CPU
CPU clock frequency	1.60 GHz
Architecture	x86_64
CPU cores / threads	4 cores / 8 threads
Operating system	Linux
Instruction-set usage	General 64-bit instructions; no AVX2 or SIMD-specific implementation
External cryptographic library	Not reported
Timing method	<code>rdtscp</code> with <code>lfence</code>
Threading	Single-threaded
Raw timing counter	CPU cycles
Reported performance unit	10^6 CPU cycles

By the public signature-coordinate convention, the signer outputs $\sigma = \Pi_{\text{pub}}(y) \in \mathbb{F}_q^N$. Hence $\Pi_{\text{pub}}^{-1}(\sigma) = y$. By construction of the residual coefficient vector in Origami.KeyGen and by the public variable-permutation convention, the expanded public MQ map reconstructed from the compressed public key satisfies

$$P_{\text{pub}}(\sigma) = G(\Pi_{\text{pub}}^{-1}(\sigma)). \quad (70)$$

Therefore, for the signature output by the signer,

$$P_{\text{pub}}(\sigma) = G(\Pi_{\text{pub}}^{-1}(\sigma)) = G(y) = t. \quad (71)$$

The verification algorithm reconstructs the same expanded public MQ map P_{pub} and the same public variable permutation Π_{pub} from the compressed public key. It also recomputes the same message digest $h_\mu = H_{\text{msg}}(\mu)$ and the same target vector $t = \text{HashToFieldVector}(\text{target} \parallel h_\mu \parallel \text{salt}, M, \mathbb{F}_q)$. It then evaluates $z = P_{\text{pub}}(\sigma)$ and returns 1 exactly when $z = t$. Since $P_{\text{pub}}(\sigma) = t$, every successfully generated signature is accepted. $\square$

3 Detailed performance analysis

3.1 Implementation Setup

All experiments were conducted on a mainstream 64-bit PC processor in a single-threaded setting. The implementation includes two C code paths: a portable ISO C reference path used as the baseline, and a general 64-bit optimized path using the optimized build configuration. The optimized path preserves the same algorithmic interface and data format as the reference path, but reduces scalar overhead in coefficient access, temporary storage, finite-field arithmetic, and local linear-system processing.

Both paths use the same public API, key format, signing workflow, and verification workflow. The platform and compiler configuration are listed in Table 1.

3.2 Parameter Sets and Key Sizes

In this subsection, we describe the concrete parameter sets used by the submitted Origami implementation. The four parameter instances target classical security strengths of 128, 256, 384, and 512 bits, respectively. The corresponding quantum security targets are 64, 128, 192, and 256 bits.

All submitted parameter sets are defined over $\mathbb{F}_{16}$. Therefore, each field element is encoded using four bits. The implementation uses a compact public-key representation rather than a fully dense explicit MQ representation. In particular, the public key is encoded as

$$pk = \text{param_id} \parallel \text{seed}_{pk} \parallel R_{pk},$$

where `param_id` identifies the parameter instance, `seedpk` is the public expansion seed published in the public key, and `Rpk` stores the explicit residual coefficient vector required by the compact public key.

The submitted parameter sets are organized by the pair (d, k) , where d is the number of main local zones and k is the number of local copies used in each main local zone. The concrete flattened dimensions used by the implementation are

$$N = 32dk + 8, \quad M = 16dk + 8.$$

The additional 8 coordinates come from the tail block used in the submitted implementation. These formulas replace the older uniform-block study based on $M = 16d$ and $N = 32d$.

For a target pair (n_c, n_q) , the parameter set is required to provide sufficient margin against the known attack classes considered in the security analysis. In particular, the loss terms, hidden-structure recovery cost, and direct or multi-target MQ inversion cost should remain above the corresponding target levels. In this subsection, we therefore list the concrete submitted parameter instances and their derived storage sizes, rather than estimating the size of a fully dense explicit MQ public key.

The seed and salt lengths are chosen according to the target level. For the four submitted parameter sets, the byte lengths satisfy

$$|\text{seed}_{sk}| = |\text{seed}_{pk}| = |\text{salt}| \in \{16, 32, 48, 64\} \text{ bytes},$$

corresponding to 128, 256, 384, and 512 bits, respectively. The compact secret key stores only the master secret expansion seed, while the derived public expansion seed is published as part of the compact public key. The parameter μ_{red} denotes the reduction parameter used by the submitted implementation, corresponding to the code constant `ORIGAMI_MU_RED`.

Table 2 – Concrete Origami parameter sets used by the submitted implementation.

Scheme	Classical	Quantum	q	(d, k)	(N, M)	μ_{red}	Seed/Salt
Origami-128	128	64	16	(3, 2)	(200, 104)	49	16 B
Origami-256	256	128	16	(6, 5)	(968, 488)	81	32 B
Origami-384	384	192	16	(8, 7)	(1800, 904)	114	48 B
Origami-512	512	256	16	(9, 8)	(2312, 1160)	146	64 B

Since $q = 16$, each coefficient in the residual coefficient vector occupies four bits. Hence the explicit residual coefficient vector R_{pk} occupies

$$|R_{\text{pk}}| = \frac{1}{2}|\mathcal{I}_{\text{res}}| \text{ bytes.}$$

The compact public-key size is therefore

$$|pk| = 4 + \kappa_{\text{pk}}/8 + |R_{\text{pk}}|,$$

where the first term is the four-byte parameter identifier, the second term is the public expansion seed, and the third term is the explicit residual coefficient vector. The residual sizes and the resulting compact public-key lengths are listed in Table 3.

Table 3 – Public-key residual size of the submitted Origami parameter sets.

Scheme	$ \mathcal{I}_{\text{res}} $	R_{pk} size	seed _{pk}	param_id	Public key
Origami-128	5,952	2,976 B	16 B	4 B	2,996 B
Origami-256	29,864	14,932 B	32 B	4 B	14,968 B
Origami-384	55,776	27,888 B	48 B	4 B	27,940 B
Origami-512	71,712	35,856 B	64 B	4 B	35,924 B

The compact secret key stores only the master secret expansion seed. Therefore, the secret-key length is

$$|sk| = \kappa_{\text{sk}}/8 \text{ bytes.}$$

A signature consists of the encoded public signature vector and the salt:

$$\text{sig} = \text{EncodeVec}_{16}(\sigma) \parallel \text{salt}, \quad \sigma \in \mathbb{F}_{16}^N.$$

Since each element of $\mathbb{F}_{16}$ occupies four bits, the signature size is

$$|\text{sig}| = \left\lceil \frac{N}{2} \right\rceil + \lambda_{\text{salt}}/8 \text{ bytes.}$$

Combining the compact secret-key size, the compact public-key size, and the signature size gives the storage summary in Table 4.

Table 4 – Key sizes of the submitted Origami parameter sets.

Scheme	Secret key	Public key	Signature
Origami-128	16 B	2.996 KB	116 B
Origami-256	32 B	14.968 KB	516 B
Origami-384	48 B	27.940 KB	948 B
Origami-512	64 B	35.924 KB	1,220 B

Compared with dense explicit MQ publication, the submitted compact public-key representation avoids storing all quadratic coefficients of the expanded public MQ map. In a dense representation, the number of stored coefficients would grow with the number of output equations and the number of quadratic monomials in the public variables. Such a representation is useful as a conceptual verification model, but it is not the submitted public-key format. In the submitted implementation, the verifier reconstructs the expanded public MQ map from the compact public key before evaluation.

The compact public-key size is therefore governed by the number of residual coefficient positions rather than by the full dense MQ coefficient array. The public expansion seed generates the seed-defined part of the public coefficient array, while only the residual coefficient vector R_{pk} is stored explicitly. Since $q = 16$, each residual coefficient occupies four bits, and hence the storage cost of R_{pk} is exactly one half of $|\mathcal{I}_{\text{res}}|$ bytes. This is why Table 3 reports the residual-position count together with the resulting public-key length.

The four submitted parameter sets increase both the number of main local zones and the number of local copies as the target level grows. This differs from the older uniform-block study, where the parameter growth was described only by increasing the number of zones. In the submitted implementation, the dimensions follow $N = 32dk + 8$, $M = 16dk + 8$, so the growth of (d, k) directly determines the size of the public signature vector, the public target vector, the residual coefficient vector, and the cost of public MQ evaluation.

The storage figures in Table 4 should be read as concrete serialization sizes for the submitted implementation. They do not include temporary buffers, expanded secret local data, cached public coefficients, or implementation-specific workspaces. In particular, the compact secret key stores only the master secret expansion seed, while the hidden local representations, hidden local algebras, private oil-coordinate partitions, and signable internal representation are reconstructed deterministically during signing.

These parameter sizes are not, by themselves, a complete proof of concrete security. The final security level of each parameter set still depends on the attack analysis in Section 4, including the restart bound, the bad-set density, hidden-structure recovery attacks, direct and hybrid MQ solving attacks, and multi-target inversion attacks. The purpose of this subsection is therefore to fix the concrete submitted parameter instances and their serialized key and signature sizes before moving to the implementation measurements in the following section.

3.3 Communication and Storage Cost

This subsection summarizes the communication and storage cost derived from the serialized key and signature sizes reported in the previous subsection. To avoid repeating

Table 5 – Communication and storage cost of all Origami instances.

Scheme	Signature transmission	Public key + signature	Signer storage	Key-pair storage
Origami-128	116 B	3,112 B	16 B	3,012 B
Origami-256	516 B	15,484 B	32 B	15,000 B
Origami-384	948 B	28,888 B	48 B	27,988 B
Origami-512	1,220 B	37,144 B	64 B	35,988 B

the parameter-set and key-size discussion, we only consider the actual transmission and storage scenarios relevant to deployment.

For communication cost, we consider two cases. In the first case, the verifier has already obtained and cached the public key. Then each authentication or verification instance only needs to transmit the signature, and the communication cost is

$$C_{\text{sig}} = |\text{sig}|_{\text{bytes}}. \quad (72)$$

In the second case, the public key is transmitted together with the signature. This gives the one-time public-key-and-signature transmission cost

$$C_{\text{pk+sig}} = |\text{pk}|_{\text{bytes}} + |\text{sig}|_{\text{bytes}}. \quad (73)$$

For storage cost, the signer only needs to store the compact secret key, while the verifier stores the compressed public key. If both keys are counted together, the serialized key-pair storage cost is

$$C_{\text{keypair}} = |\text{pk}|_{\text{bytes}} + |\text{sk}|_{\text{bytes}}. \quad (74)$$

These costs refer only to serialized objects. They do not include temporary buffers, expanded public coefficients, expanded secret local data, cached local matrices, or other implementation-specific workspaces.

Table 5 reports the resulting communication and storage costs for all submitted Origami instances. The results show that the repeated communication cost is dominated by the signature when the public key has already been cached by the verifier. In this cached-public-key case, the transmission size ranges from 116 bytes for Origami-128 to 1,220 bytes for Origami-512. When the public key must be transmitted together with the signature, the one-time transmission size increases to 3,112 bytes, 15,484 bytes, 28,888 bytes, and 37,144 bytes for the four instances, respectively.

To further relate the byte-level communication cost to transmission-time overhead, let R_{net} denote the effective application-layer transmission rate in Mbit/s and let S denote the transmitted payload size in bytes. The payload transmission time is

$$T_{\text{tx}}(S, R_{\text{net}}) = \frac{8S}{R_{\text{net}}} \mu\text{s}. \quad (75)$$

Under the experimental CPU configuration reported above, the nominal CPU frequency is 1.60 GHz. Therefore, the same transmission time corresponds to

$$C_{\text{tx}}^{(10^6 \text{ cycles})}(S, R_{\text{net}}) = 1.6 \times 10^{-3} T_{\text{tx}}(S, R_{\text{net}}) = \frac{0.0128S}{R_{\text{net}}}, \quad (76)$$

Table 6 – Estimated payload transmission-time overhead of all Origami instances.

Scheme	100 Mbit/s		1 Gbit/s	
	Signature	Public key + signature	Signature	Public key + signature
Origami-128	9.28 μ s	248.96 μ s	0.93 μ s	24.90 μ s
Origami-256	41.28 μ s	1.24 ms	4.13 μ s	123.87 μ s
Origami-384	75.84 μ s	2.31 ms	7.58 μ s	231.10 μ s
Origami-512	97.60 μ s	2.97 ms	9.76 μ s	297.15 μ s

where $C_{tx}^{(10^6 \text{ cycles})}$ is measured in units of 10^6 CPU cycles. This conversion is used only to compare the communication delay with the timing unit used in the implementation experiments. It does not include protocol headers, propagation delay, queuing delay, operating-system network-stack overhead, memory copy overhead, or retransmission cost.

Table 6 reports the estimated payload transmission time for two representative effective link rates. When the public key has been cached by the verifier, the repeated transmission overhead is small: at 100 Mbit/s, it ranges from 9.28 μ s for Origami-128 to 97.60 μ s for Origami-512. When the public key is transmitted together with the signature, the one-time payload transmission time is mainly determined by the compressed public key and reaches 2.97 ms at 100 Mbit/s for Origami-512. At 1 Gbit/s, the corresponding one-time payload transmission time of Origami-512 is reduced to 297.15 μ s.

On the storage side, the signer-side storage remains very small because the compact secret key is only the master secret expansion seed. The signer stores 16 bytes, 32 bytes, 48 bytes, and 64 bytes for Origami-128, Origami-256, Origami-384, and Origami-512, respectively. The key-pair storage is therefore almost completely determined by the compressed public key. This indicates that the main deployment-side storage cost is verifier-side public-key storage, while signer-side storage remains negligible for all submitted instances.

Overall, the communication and storage results show a clear deployment pattern. In repeated authentication scenarios, where the verifier caches the public key, Origami only transmits the signature and therefore has low per-session communication overhead. In initial enrollment or public-key distribution scenarios, the public key dominates the one-time transmission and storage cost. The transmission-time estimates show that even the largest one-time payload remains at the millisecond level on a 100 Mbit/s link and below the sub-millisecond level on a 1 Gbit/s link, while the repeated signature-only transmission remains much smaller for all instances.

3.4 Implementation performance

We next report the performance of the reference implementation and the general 64-bit PC optimized implementation. The reference implementation is used to measure the direct cost of the algorithmic workflow without platform-specific vector instructions. The optimized implementation uses the same compressed key format and the same algorithmic interface, but it is compiled with the `ORIGAMI_OPTIMIZED_PROFILE` profile. Its goal is to reduce scalar overheads in coefficient access, temporary storage, finite-field arithmetic, and local linear-system evaluation, without relying on AVX2 or SIMD-specific instructions.

Table 7 – Reference implementation performance of Origami (10^6 cycles). Each entry is mean (median/standard deviation).

Scheme	KeyGen		Sign		Verify
Origami-128	3.36	(3.24/0.24)	35.33	(33.27/7.55)	27.75 (27.78/1.33)
Origami-256	100.12	(103.12/22.22)	1691.97	(1570.40/249.36)	1540.41 (1536.43/27.02)
Origami-384	274.77	(241.71/47.53)	7687.51	(7615.80/185.88)	7494.42 (7455.83/249.30)
Origami-512	425.50	(425.50/2.13)	15345.37	(14496.50/1372.08)	14036.05 (13996.05/108.41)

Table 7 shows that key generation is much cheaper than signing and verification in the reference implementation. For Origami-128, key generation takes 3.36 million cycles on average, while signing and verification take 35.33 and 27.75 million cycles, respectively. For Origami-512, the same three operations increase to 425.50, 15345.37, and 14036.05 million cycles.

The main growth appears in signing and verification. From Origami-128 to Origami-512, the reference key generation cost increases by about $126.6\times$, while the signing and verification costs increase by about $434.3\times$ and $505.8\times$, respectively. This is consistent with the structure of Origami: signing and verification both require large-scale finite-field MQ evaluation over the expanded public dimensions, and signing additionally performs local-zone sequential linear solving.

Signing is slightly more expensive than verification for all four parameter sets. In the reference implementation, the signing-to-verification ratio is about $1.27\times$ for Origami-128, $1.10\times$ for Origami-256, $1.03\times$ for Origami-384, and $1.09\times$ for Origami-512. This gap reflects the extra work caused by local sampling, rank checks, and solving the local linear systems during signature generation.

Table 8 – Optimized implementation performance of Origami (10^6 cycles). Each entry is mean (median/standard deviation).

Scheme	Runs	KeyGen		Sign		Verify
Origami-128	10/20/30	2.32	(2.18/0.36)	34.79	(33.14/6.06)	32.84 (32.97/2.35)
Origami-256	5/10/20	90.59	(74.43/22.22)	1701.50	(1596.50/244.70)	1557.15 (1548.81/31.20)
Origami-384	3/5/10	261.63	(249.18/22.05)	7766.47	(7871.43/259.09)	7383.31 (7386.56/117.48)
Origami-512	2/3/5	429.48	(429.48/28.91)	15296.44	(14306.29/1437.50)	14198.53 (13974.08/406.19)

Table 8 shows that the optimized implementation preserves the same overall scaling trend as the reference implementation. For Origami-128, the optimized implementation requires 2.32 million cycles for key generation, 34.79 million cycles for signing, and 32.84 million cycles for verification. For Origami-512, these values increase to 429.48, 15296.44, and 14198.53 million cycles, respectively.

The optimized key-generation cost is lower than the reference key-generation cost for Origami-128, Origami-256, and Origami-384. This suggests that the optimized path is useful for reducing some scalar overheads related to coefficient handling and public-key data preparation. For Origami-512, however, the optimized key-generation cost is slightly higher than the reference value. This indicates that the current optimized path does not provide a uniform advantage at all parameter sizes.

For signing and verification, the optimized implementation gives only limited changes. Both operations remain dominated by expanded public MQ evaluation and local-zone linear-system processing. In particular, signing still needs to sample the current local

variables, evaluate the local matrix, check its rank, and solve for the solved part W_j . Verification still reconstructs and evaluates the expanded public MQ map P_{pub} . Hence, the optimized path changes the constant factors of the general C implementation, but it does not remove the main arithmetic bottlenecks.

For a direct comparison, we use the ratio between the reference mean cycle count and the optimized mean cycle count:

$$\text{ratio} = \frac{\text{mean cycles of reference}}{\text{mean cycles of optimized}}.$$

A ratio larger than 1 means that the optimized implementation is faster, while a ratio close to 1 means that the two implementation paths have similar performance.

Table 9 – Reference-to-optimized performance ratios.

Scheme	KeyGen ratio	Sign ratio	Verify ratio
Origami-128	1.45x	1.02x	0.84x
Origami-256	1.11x	0.99x	0.99x
Origami-384	1.05x	0.99x	1.02x
Origami-512	0.99x	1.00x	0.99x

Table 9 confirms that the main visible benefit of the optimized path appears in key generation for the smaller and medium parameter sets. The key-generation ratio is $1.45\times$ for Origami-128, $1.11\times$ for Origami-256, and $1.05\times$ for Origami-384. For Origami-512, the ratio is $0.99\times$, so the two paths are essentially tied.

For signing and verification, most ratios stay close to 1. The optimized signing ratios are $1.02\times$, $0.99\times$, $0.99\times$, and $1.00\times$ from Origami-128 to Origami-512. The optimized verification ratios are $0.84\times$, $0.99\times$, $1.02\times$, and $0.99\times$. These results show that the current optimized C path does not substantially change the dominant cost of signing and verification. The main reason is that these two operations are still controlled by finite-field MQ evaluation and local-zone linear-system processing.

3.5 Restart behavior performance

We finally report the restart statistics of the signing algorithm. A restart occurs when the local matrix in a local zone does not have full row rank, so the signer resamples the current local randomness and tries again. The rank failure rate is computed as

$$\text{rank failure rate} = \frac{\text{number of local rank failures}}{\text{number of evaluated local systems}}.$$

Table 10 – Restart behavior of the signing algorithm.

Scheme	Sign runs	Avg. restarts	Max restarts	Rank failure rate	Avg. local systems
Origami-128	20	0.250	1	5.8824%	4.250
Origami-256	10	0.400	2	5.4054%	7.400
Origami-384	5	0.200	1	2.1739%	9.200
Origami-512	3	0.333	1	3.2258%	10.333

Table 10 shows that the average number of restarts is below 1 for all four parameter sets, and the maximum number of observed restarts is at most 2. The average number of local systems is also consistent with the number of local zones plus the observed restart count. For example, Origami-128 uses $d = 3$, so a signing attempt without restart evaluates $d + 1 = 4$ local systems; the measured average is 4.250, matching the average restart count 0.250. Similarly, Origami-512 uses $d = 9$, so the no-restart baseline is 10 local systems, and the measured average is 10.333.

The rank failure rate remains between 2.1739% and 5.8824%. These values indicate that local rank failures are observable in the implementation but do not cause large instability in the signing time. Therefore, the growth of signing cost is mainly caused by the increasing number and size of local systems, rather than by excessive restart behavior.

Overall, the submitted Origami implementation completes key generation, signing, and verification for all four tested parameter sets under both implementation paths. As the parameter set grows from Origami-128 to Origami-512, the public dimensions, residual-position set, explicit residual coefficient vector, public key size, signature size, and running cost all increase. The reference implementation provides a direct baseline for the algorithmic workflow. The optimized implementation improves key generation for Origami-128, Origami-256, and Origami-384, but it has only a limited effect on signing and verification. Thus, the main implementation bottlenecks are expanded public MQ evaluation, finite-field arithmetic, coefficient access, and local linear-system solving.

4 Security Analysis

In this section, we analyze the security of Origami. We first define the computational problems underlying the design. We then prove a reduction from the EUF-CMA security of Origami in the random oracle model to these problems. Finally, we discuss the explicit loss terms appearing in the reduction.

In the compact-key implementation analyzed in this section, the public key is not stored as a full dense coefficient list of the public MQ map. It is stored as

$$pk = \text{param_id} \parallel \text{seed}_{pk} \parallel R_{pk}. \quad (77)$$

The current construction does not instantiate external input/output linear hiding maps T_{in} and T_{out} . Instead, the verifier reconstructs the expanded public MQ map from the public expansion seed, the explicit residual coefficient vector, and the public residual-position set:

$$P_{pub} = P_{seed} + \text{Embed}_{I_{res}}(R_{pk}), \quad P_{pub} : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^M. \quad (78)$$

In the submitted implementation, all domain-separated expansions used in this security analysis are instantiated by the same SHAKE256 absorption convention. For a seed `seed`, an ASCII domain label `label`, three 32-bit domain parameters a, b, c , an optional extra string `extra`, and an output length L in bytes, the implementation absorbs `seed` $\parallel$ ASCII(`label`) $\parallel$ u32(a) $\parallel$ u32(b) $\parallel$ u32(c) $\parallel$ `extra` into SHAKE256 and then squeezes L output bytes. Equivalently,

$$\begin{aligned} & \text{Expand}(\text{seed}, \text{label}, a, b, c, \text{extra}, L) \\ &= \text{SHAKE256}(\text{seed} \parallel \text{ASCII}(\text{label}) \parallel \text{u32}(a) \parallel \text{u32}(b) \parallel \text{u32}(c) \parallel \text{extra}) \big|_{L \text{ bytes}}. \end{aligned} \quad (79)$$

The string `extra` may be empty. This convention is used for the public seed derivation, coefficient-seed derivation, residual-coefficient expansion, hidden local algebra sampling, seed-defined public affine coefficients, signing known-coordinate sampling, and kernel sampling. Thus, expressions such as `Expand(seed, label || a || b || c, L)` should be read as this fixed implementation-level domain-separated SHAKE256 expansion. If $G : \mathbb{F}_q^N \rightarrow \mathbb{F}_q^M$ denotes the flattened signable map induced by the internal local-zone construction, then the public coordinate convention is

$$P_{\text{pub}}(\sigma) = G(\Pi_{\text{pub}}^{-1}(\sigma)). \quad (80)$$

The internal central map contains d main local zones and one tail zone:

$$F = (F^{(1)}, \dots, F^{(d+1)}). \quad (81)$$

Accordingly, the public input and output dimensions are

$$\begin{aligned} N &= \sum_{j=1}^{d+1} k_j(v_j + o_j)l_j^2, \\ M &= \sum_{j=1}^{d+1} M_j = \sum_{j=1}^{d+1} m_j l_j^2. \end{aligned} \quad (82)$$

For each local zone $j \in \{1, \dots, d+1\}$ and each local copy $i \in \{1, \dots, k_j\}$, let

$$K_{j,i} \subseteq \text{Mat}_{l_j}(\mathbb{F}_q) \quad (83)$$

denote the hidden local algebra used internally by the signer. We write

$$\begin{aligned} Y &= (Y^{(1)}, \dots, Y^{(d+1)}), \\ Y^{(j)} &= (Y_1^{(j)}, \dots, Y_{k_j}^{(j)}), \end{aligned} \quad (84)$$

where

$$Y_i^{(j)} = (V_i^{(j)}, O_i^{(j)}) \in K_{j,i}^{v_j} \times K_{j,i}^{o_j}. \quad (85)$$

We also write

$$Y^{(<j)} = (Y^{(1)}, \dots, Y^{(j-1)}) \quad (86)$$

for the prefix of local zones that have already been fixed before zone j is solved.

The flattening map is

$$\text{Flat} : \prod_{j=1}^{d+1} \prod_{i=1}^{k_j} K_{j,i}^{v_j+o_j} \rightarrow \mathbb{F}_q^N. \quad (87)$$

The corresponding output flattening map is

$$\text{FlatOut} : \prod_{j=1}^{d+1} \mathbb{F}_q^{M_j} \rightarrow \mathbb{F}_q^M. \quad (88)$$

For local zone j , the flattened oil-coordinate vector is denoted by

$$Z_j \in \mathbb{F}_q^{k_j o_j l_j^2}. \quad (89)$$

The secret key determines a private oil-coordinate partition

$$\rho_j : Z_j \mapsto (U_j, W_j), \quad U_j \in \mathbb{F}_q^{a_j}, \quad W_j \in \mathbb{F}_q^{b_j}. \quad (90)$$

Before solving local zone j , the signer has already fixed the prefix $Y^{(<j)}$, the current-zone vinegar variables, and the sampled part U_j . We write

$$X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j). \quad (91)$$

The j -th local zone map has the affine form

$$F^{(j)}(Y^{(\leq j)}) = D_j(X_j) + A_j(X_j)W_j, \quad (92)$$

where

$$D_j(X_j) \in \mathbb{F}_q^{M_j}, \quad A_j(X_j) \in \mathbb{F}_q^{M_j \times b_j}. \quad (93)$$

Thus, for a target block $u^{(j)} \in \mathbb{F}_q^{M_j}$, signing in local zone j reduces to the rectangular linear system

$$A_j(X_j)W_j = u^{(j)} - D_j(X_j). \quad (94)$$

The implementation succeeds on this local zone when

$$\text{rank}(A_j(X_j)) = M_j. \quad (95)$$

No quadratic term in W_j is allowed, and no term may depend on future-zone variables. This affine restriction is the property that supports sequential linear solving.

The target vector is derived directly in the public target space. The message is first compressed to the fixed-length digest

$$h_\mu = H_{\text{msg}}(\mu), \quad (96)$$

and the target vector is then computed as

$$t = \text{HashToFieldVector}(\text{target} \parallel h_\mu \parallel \text{salt}, M, \mathbb{F}_q). \quad (97)$$

It is split according to the local-zone output dimensions as

$$t = (u^{(1)}, \dots, u^{(d+1)}), \quad u^{(j)} \in \mathbb{F}_q^{M_j}. \quad (98)$$

No public output hiding is undone during signing; the target blocks $u^{(j)}$ are used directly in the local-zone equations of the reconstructed signable internal representation. We model this target-derivation procedure as a random oracle into $\mathbb{F}_q^M$, and we denote the salt length by λ_{salt} bits.

Definition 2 (EUF-CMA security) *Let the target-derivation procedure be modeled as a random oracle, and let $\mathcal{A}$ be an adversary. In the EUF-CMA experiment, the challenger runs KeyGen to generate (pk, sk) , gives the compact public key pk to $\mathcal{A}$, and grants oracle access to the target-derivation random oracle and to the signing oracle $\text{Sign}(sk, \cdot)$. The adversary outputs a message-signature pair $(\mu, (\sigma, \text{salt}))$ and succeeds if*

$$\text{Verify}(pk, \mu, (\sigma, \text{salt})) = 1 \quad (99)$$

and μ was not queried to the signing oracle. The corresponding advantage is denoted by

$$\text{Adv}_{\text{Origami}}^{\text{EUF-CMA}}(\mathcal{A}). \quad (100)$$

Definition 3 (EUF-KOA security) *Let the target-derivation procedure be modeled as a random oracle, and let $\mathcal{A}$ be an adversary. In the EUF-KOA experiment, the challenger runs **KeyGen** to generate (pk, sk) , gives the compact public key pk to $\mathcal{A}$, and grants oracle access to the target-derivation random oracle, but no signing oracle access. For a message μ , the oracle input is interpreted through the fixed message digest*

$$h_\mu = H_{\text{msg}}(\mu) \quad (101)$$

and the target-domain input

$$\text{target} \parallel h_\mu \parallel \text{salt}. \quad (102)$$

The adversary outputs a message-signature pair $(\mu, (\sigma, \text{salt}))$ and succeeds if

$$\text{Verify}(pk, \mu, (\sigma, \text{salt})) = 1. \quad (103)$$

The corresponding advantage is denoted by

$$\text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{A}). \quad (104)$$

4.1 Hard Problems

We now define the hard problems used in the security proof. The first captures the hidden-structure requirement of the compact public-key distribution used by the submitted Origami implementation.

Definition 4 (HP-OV problem) *Let $D_{\text{Origami}}^{\text{comp}}$ denote the distribution of expanded public maps induced by compact Origami public keys generated by **KeyGen**. A sample from this distribution is obtained by sampling the master secret seed, deriving seed_{pk} and the coefficient seed, computing $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$, generating the seed-defined coefficient part P_{seed} , generating the explicit residual coefficient vector R_{pk} , and reconstructing $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$ under the public coordinate convention induced by Π_{pub} .*

In the submitted implementation, the residual-selection rule is instantiated by a public stride-and-offset schedule over the ordered eligible-position universe. Let $T_{\text{elig}} = |I_{\text{elig}}|$. The implementation computes a public stride $s = \text{Stride}(\text{param_id}, T_{\text{elig}})$ satisfying $\gcd(s, T_{\text{elig}}) = 1$, and a public offset $o = \text{Offset}(\text{seed}_{\text{pk}}, \text{param_id}, T_{\text{elig}})$. For an eligible coefficient position with canonical linear index $idx \in \{0, \dots, T_{\text{elig}} - 1\}$, define $\text{rank}(idx) = (idx \cdot s + o) \bmod T_{\text{elig}}$. Then $idx \in I_{\text{res}}$ if and only if $\text{rank}(idx) < |I_{\text{res}}|$, and the corresponding residual coefficient is $R_{\text{pk}}[\text{rank}(idx)]$. This residual-position schedule is public and deterministic. It depends only on the public parameter identifier, the public expansion seed, and the fixed eligible-position ordering. It does not use secret information and does not reveal the private sampled/solved split or the private oil-coordinate partitions.

Let $D_{\text{elig}}^{\text{rand}}$ denote the reference distribution defined with the same public parameters, the same residual-eligible affine coefficient-position universe I_{elig} , the same residual-selection rule, the same encoding format, and the same public coordinate convention, but

with the corresponding eligible-affine public coefficients sampled independently and uniformly over $\mathbb{F}_q$. The HP-OV problem is to distinguish a public map sampled from $D_{\text{Origami}}^{\text{comp}}$ from a public map sampled from $D_{\text{elig}}^{\text{rand}}$.

For an adversary $\mathcal{B}$, define

$$\begin{aligned} \text{AdvHP-OV}_{\text{Origami}}(\mathcal{B}) = & \left| \Pr[\mathcal{B}(P_{\text{pub}}) = 1 \mid P_{\text{pub}} \leftarrow D_{\text{Origami}}^{\text{comp}}] \right. \\ & \left. - \Pr[\mathcal{B}(P_{\text{pub}}) = 1 \mid P_{\text{pub}} \leftarrow D_{\text{elig}}^{\text{rand}}] \right|. \end{aligned} \quad (105)$$

Assumption 5 (HP-OV hidden-structure indistinguishability) *For every efficient adversary $\mathcal{B}$, the quantity $\text{AdvHP-OV}_{\text{Origami}}(\mathcal{B})$ is negligible.*

This is the hidden-structure assumption for the compact public-key version of Origami. It does not require the expanded public map to be statistically identical to a fully dense uniform MQ map. Rather, it states that the compactly generated public map, together with the public residual-position set, the explicit residual coefficients, and the public coordinate permutation, does not reveal an efficient way to recover the private oil-coordinate partitions, the sampled/solved split, the local zone boundaries, or the hidden local algebras.

Our second assumption concerns inversion once the compact public-key distribution has been replaced by the eligible-affine random reference distribution.

Definition 6 (MT-MQ problem) *Let $P_{\text{pub}} \leftarrow D_{\text{elig}}^{\text{rand}}$ be a public MQ map sampled from the eligible-affine random reference distribution defined in Definition 4. Let $\{t_i\}_{i \in \mathbb{N}} \leftarrow (\mathbb{F}_q^M)^{\mathbb{N}}$ be an infinite sequence of independent uniform targets. The MT-MQ problem is to output a pair (I, σ) such that $P_{\text{pub}}(\sigma) = t_I$.*

For an adversary $\mathcal{B}'$, the advantage is defined by

$$\text{AdvMT-MQ}_{N,M,q}(\mathcal{B}') = \Pr[P_{\text{pub}}(\sigma) = t_I], \quad (106)$$

where the probability is taken over $P_{\text{pub}} \leftarrow D_{\text{elig}}^{\text{rand}}$, $\{t_i\}_{i \in \mathbb{N}} \leftarrow (\mathbb{F}_q^M)^{\mathbb{N}}$, and $(I, \sigma) \leftarrow \mathcal{B}'(P_{\text{pub}}; \{t_i\}_{i \in \mathbb{N}})$.

Assumption 7 (MT-MQ multi-target inversion hardness) *For every efficient adversary $\mathcal{B}'$, the quantity $\text{AdvMT-MQ}_{N,M,q}(\mathcal{B}')$ is negligible.*

After the compact public-key structure has been abstracted away through Definition 4 and Assumption 5, the remaining hardness is expressed by the MT-MQ inversion problem over the eligible-affine random reference distribution. This assumption captures the generic inversion layer left after no exploitable information about the private oil-coordinate partitions, the sampled/solved split, the local zone boundaries, or the hidden local algebras is available. Besides these two computational assumptions, the proof uses two technical distributional assumptions. They do not assert computational hardness; they are used only to relate the real signing distribution to the simulated distributions appearing in the reduction.

We define the good-signature set for the expanded public map P_{pub} by

$$\begin{aligned} \text{GoodSig}_{P_{\text{pub}}} = \left\{ \sigma \in \mathbb{F}_q^N \mid \begin{array}{l} \text{there exists an internal assignment } Y = (Y^{(1)}, \dots, Y^{(d+1)}) \\ \text{such that } \sigma = \Pi_{\text{pub}}(\text{Flat}(Y)), \\ \text{and for every } j \in \{1, \dots, d+1\}, \\ \text{if } \rho_j(Z_j) = (U_j, W_j) \text{ and } X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j), \\ \text{then } \text{rank}(A_j(X_j)) = M_j \end{array} \right\}. \end{aligned} \quad (107)$$

Assumption 8 (Good-signature uniformity) *For every fresh signing query, conditioned on the event that one ideal signing attempt succeeds, the output distribution of the signer is equal to the distribution obtained by sampling σ uniformly from $\text{GoodSig}_{P_{\text{pub}}}$ and then programming the target-derivation random oracle at target $\|h_\mu\|$ salt to the corresponding target value $P_{\text{pub}}(\sigma)$.*

In the submitted implementation, the sampled coordinates in U_j are generated from a domain-separated SHAKE stream. For each local zone with $a_j > 0$, the first sampled coordinate of U_j is normalized by the rule

$$u_{j,0} = \begin{cases} 1, & \tilde{u}_{j,0} = 0, \\ \tilde{u}_{j,0}, & \tilde{u}_{j,0} \neq 0, \end{cases} \quad \text{for } a_j > 0. \quad (108)$$

The value $\tilde{u}_{j,0}$ is the first field element produced by the corresponding domain-separated sampling stream. For the remaining sampled coordinates, $u_{j,r} = \tilde{u}_{j,r}$ for $1 \leq r < a_j$. Zones with $a_j = 0$ have no sampled U_j coordinate and therefore do not apply this normalization. Thus, the proof uses an idealized notation for sampled coordinates, while the concrete signing distribution considered by $\text{GoodSig}_{P_{\text{pub}}}$ and by the restart statistics includes this implementation-level nonzero normalization.

Assumption 9 (Bad-set density bound) *There exists a parameter-dependent bound B_{bad} such that, for uniformly random $\sigma \leftarrow \mathbb{F}_q^N$, $\Pr[\sigma \notin \text{GoodSig}_{P_{\text{pub}}}] \leq B_{\text{bad}}$.*

Remark 10 *Assumptions 8 and 9 are used only to justify the transition from the real signing distribution to the programmed-random-oracle distribution. They are distributional regularity assumptions rather than additional computational hardness assumptions. If a finer analysis of the signing distribution and the full-row-rank local systems becomes available, they may be replaced by explicit lemmas.*

4.2 Security Proof

We now prove the main theorem. The proof has two steps: first, we reduce EUF-CMA security to EUF-KOA security; then, we reduce EUF-KOA security to the HP-OV and MT-MQ problems.

Since signing may restart when the current local matrix does not have full row rank, we first record the restart bound.

Definition 11 (Zone failure probability) For each local zone $j \in \{1, \dots, d+1\}$, let

$$\beta_j = \max_{Y^{(<j)}} \Pr_{V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j} [\text{rank}(A_j(X_j)) < M_j], \quad X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j). \quad (109)$$

The failure bound for one ideal signing attempt is

$$B_{\text{Origami}} = 1 - \prod_{j=1}^{d+1} (1 - \beta_j) \leq \sum_{j=1}^{d+1} \beta_j. \quad (110)$$

Lemma 12 (Restart bound) One ideal Origami signing attempt restarts with probability at most B_{Origami} .

Proof. For any fixed prefix $Y^{(<j)}$, local zone j fails exactly when $\text{rank}(A_j(X_j)) < M_j$. By the definition of β_j , this event has probability at most β_j . Hence all local zones succeed with probability at least $\prod_{j=1}^{d+1} (1 - \beta_j)$, and the restart probability is at most $1 - \prod_{j=1}^{d+1} (1 - \beta_j) = B_{\text{Origami}}$. $\square$

Lemma 13 (Conditioned signing distribution) The output distribution of the ideal repeat-until-success signer is the distribution of one ideal signing attempt conditioned on success.

Proof. Let E be the event that one ideal signing attempt succeeds, let $\bar{E}$ be its complement, and write $p_s = \Pr[E]$ and $p_f = \Pr[\bar{E}] = 1 - p_s$. Let X be the output of one successful attempt and let D_{succ} be the distribution of X conditioned on E . For any set S of signatures, the repeat-until-success signer outputs an element of S with probability

$$\sum_{r \geq 0} p_f^r \Pr[E \cap \{X \in S\}] = \frac{\Pr[E \cap \{X \in S\}]}{1 - p_f} = \frac{\Pr[E \cap \{X \in S\}]}{p_s} = \Pr_{D_{\text{succ}}} [S]. \quad (111)$$

Thus the repeat-until-success output distribution is exactly the success-conditioned one-attempt distribution. $\square$

The submitted implementation uses the finite per-zone bound `MAX_SIGN_ATTEMPTS` = 8192. Failure within this bound is treated as an implementation-level signing abort. The theorem below analyzes the ideal repeat-until-success signer; the finite-attempt abort probability is a correctness event measured separately from the reduction.

Theorem 14 (EUF-CMA security of Origami) Let $\mathcal{A}$ be a EUF-CMA adversary against Origami in the random oracle model. Suppose $\mathcal{A}$ runs in time T , makes at most Q_s signing queries and at most Q_h random-oracle queries. If Assumptions 5, 7, 8, and 9 hold, then there exist adversaries $\mathcal{B}$ and $\mathcal{B}'$ against HP-OV and MT-MQ, respectively, running in time at most $T + (Q_s + Q_h + 1) \cdot \text{poly}(N, M, \log q)$, such that

$$\begin{aligned} \text{AdvEUF-CMA}_{\text{Origami}}(\mathcal{A}) &\leq \text{AdvHP-OV}_{\text{Origami}}(\mathcal{B}) + \text{AdvMT-MQ}_{N,M,q}(\mathcal{B}') + q^{-M} \\ &\quad + Q_s B_{\text{bad}} + (Q_h + Q_s) Q_s 2^{-\lambda_{\text{salt}}}. \end{aligned} \quad (112)$$

Lemma 15 (EUF-CMA-to-EUF-KOA reduction) *Let $\mathcal{A}$ be a EUF-CMA adversary in the target-derivation random-oracle model. Suppose $\mathcal{A}$ runs in time T , makes at most Q_s signing queries and at most Q_h target-derivation random-oracle queries. If Assumptions 8 and 9 hold, then there exists a EUF-KOA adversary $\mathcal{C}$ running in time at most $T + O((Q_s + Q_h)\text{poly}(N, M, \log q))$ such that*

$$\text{AdvEUF-CMA}_{\text{Origami}}(\mathcal{A}) \leq \text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C}) + Q_s B_{\text{bad}} + (Q_h + Q_s) Q_s 2^{-\lambda_{\text{salt}}}. \quad (113)$$

Proof. We use a standard game sequence. Game 0 is the real EUF-CMA experiment, so $\Pr[\text{Game}_0 = 1] = \text{AdvEUF-CMA}_{\text{Origami}}(\mathcal{A})$.

In Game 1, signing queries are answered by programming the target-derivation random oracle. The game keeps a table L containing entries (x, t, σ) or $(x, t, \perp)$, where $x = \text{target} \parallel h_\mu \parallel \text{salt}$ and $h_\mu = H_{\text{msg}}(\mu)$. A fresh oracle query on x receives a uniform $t \leftarrow \mathbb{F}_q^M$ and inserts $(x, t, \perp)$. A signing query on μ samples a fresh salt, forms x , aborts only if L already contains $(x, t, \perp)$, and otherwise programs x to the target corresponding to the returned signature.

Conditioned on no abort, this is a perfect simulation of the target-derivation random oracle. The only bad event is that a fresh signing input collides with an already queried target-domain input. By the union bound,

$$\Pr[\text{Game}_1 = 1] \geq \Pr[\text{Game}_0 = 1] - (Q_h + Q_s) Q_s 2^{-\lambda_{\text{salt}}}. \quad (114)$$

In Game 2, each fresh signing query is simulated by sampling σ uniformly from $\text{GoodSig}_{P_{\text{pub}}}$, setting $t = P_{\text{pub}}(\sigma)$, and programming the random oracle at $x = \text{target} \parallel h_\mu \parallel \text{salt}$ to t . By Lemma 13 and Assumption 8, $\Pr[\text{Game}_2 = 1] = \Pr[\text{Game}_1 = 1]$.

In Game 3, the simulated signer samples $\sigma \leftarrow \mathbb{F}_q^N$ uniformly instead of sampling from $\text{GoodSig}_{P_{\text{pub}}}$. By Assumption 9, the statistical distance over all signing queries is at most $Q_s B_{\text{bad}}$, and therefore

$$\Pr[\text{Game}_2 = 1] \leq \Pr[\text{Game}_3 = 1] + Q_s B_{\text{bad}}. \quad (115)$$

Game 4 is the EUF-KOA experiment run by adversary $\mathcal{C}$. The adversary $\mathcal{C}$ receives only pk , expands it to P_{pub} , and simulates $\mathcal{A}$ exactly as in Game 3. No trapdoor is needed, since each simulated signing answer is produced by sampling a random σ and programming the oracle value to $P_{\text{pub}}(\sigma)$. If $\mathcal{A}$ outputs a valid forgery $(\mu^*, (\sigma^*, \text{salt}^*))$, then with $x^* = \text{target} \parallel H_{\text{msg}}(\mu^*) \parallel \text{salt}^*$ we have $P_{\text{pub}}(\sigma^*) = \mathcal{H}(x^*)$, which is also a valid EUF-KOA forgery. Hence $\text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C}) \geq \Pr[\text{Game}_3 = 1]$.

Combining the game transitions proves the claim. $\square$

Remark 16 *Assumption 8 is consistent with Origami's rectangular local solving structure. For fixed $X_j = (Y^{(<j)}, V_1^{(j)}, \dots, V_{k_j}^{(j)}, U_j)$ and target block $u^{(j)}$, if $\text{rank}(A_j(X_j)) = M_j$, then the local solution set*

$$\mathcal{S}_j(X_j, u^{(j)}) = \{W_j \in \mathbb{F}_q^{b_j} \mid A_j(X_j)W_j = u^{(j)} - D_j(X_j)\} = W_j^0 + \ker(A_j(X_j)) \quad (116)$$

for any particular solution W_j^0 . After sampling W_j from this affine solution space, the signer recovers $Z_j = \rho_j^{-1}(U_j, W_j)$. Conversely, for fixed X_j and any W_j , forward evaluation determines $u^{(j)} = D_j(X_j) + A_j(X_j)W_j$. Thus solving from a target and sampling a

good signature before forward evaluation are compatible views. The current implementation uses a rectangular full-row-rank system and samples from an affine solution space, rather than inverting a square system to obtain a unique oil vector.

We next reduce EUF-KOA security to the two hard problems for the compact public-key model.

Lemma 17 (EUF-KOA-to-hardness reduction) *Let $\mathcal{C}$ be a EUF-KOA adversary in the target-derivation random-oracle model. If Assumptions 5 and 7 hold, then there exist adversaries $\mathcal{B}$ and $\mathcal{B}'$ against HP-OV and MT-MQ, respectively, such that*

$$\text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C}) \leq \text{AdvHP-OV}_{\text{Origami}}(\mathcal{B}) + \text{AdvMT-MQ}_{N,M,q}(\mathcal{B}') + q^{-M}. \quad (117)$$

Proof. Game 0 is the real EUF-KOA experiment, so $\Pr[\text{Game}_0 = 1] = \text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C})$.

Game 1 replaces the compact Origami public-map distribution by the eligible-affine random reference distribution $D_{\text{elig}}^{\text{rand}}$. An HP-OV distinguisher $\mathcal{B}$ receives a challenge public map P_{pub} , gives the corresponding verification object to $\mathcal{C}$, and simulates the target-derivation random oracle. If $P_{\text{pub}} \leftarrow D_{\text{Origami}}^{\text{comp}}$, it simulates Game 0; if $P_{\text{pub}} \leftarrow D_{\text{elig}}^{\text{rand}}$, it simulates Game 1. Thus

$$\text{AdvHP-OV}_{\text{Origami}}(\mathcal{B}) = \left| \Pr[\text{Game}_0 = 1] - \Pr[\text{Game}_1 = 1] \right|. \quad (118)$$

Game 2 answers the i -th fresh target-domain query with the i -th target t_i supplied by the MT-MQ challenger. Since the sequence $\{t_i\}_{i \in \mathbb{N}}$ is independently uniform in $\mathbb{F}_q^M$, this is a perfect simulation and $\Pr[\text{Game}_1 = 1] = \Pr[\text{Game}_2 = 1]$.

The MT-MQ adversary $\mathcal{B}'$ receives P_{pub} and the target sequence $\{t_i\}_{i \in \mathbb{N}}$, runs $\mathcal{C}$, and answers fresh target queries with the next target. If $\mathcal{C}$ outputs $(\mu^*, (\sigma^*, \text{salt}^*))$, let $x^* = \text{target} \parallel H_{\text{msg}}(\mu^*) \parallel \text{salt}^*$. If x^* was previously queried, then $\mathcal{B}'$ knows the corresponding index I ; validity gives $P_{\text{pub}}(\sigma^*) = t_I$, so $\mathcal{B}'$ outputs (I, σ^*) .

If x^* was never queried, then $\mathcal{H}(x^*)$ is fresh uniform in $\mathbb{F}_q^M$ from the adversary's view, and

$$\Pr[P_{\text{pub}}(\sigma^*) = \mathcal{H}(x^*) \text{ and } x^* \text{ was never queried}] \leq q^{-M}. \quad (119)$$

Therefore

$$\Pr[\text{Game}_2 = 1] \leq \text{AdvMT-MQ}_{N,M,q}(\mathcal{B}') + q^{-M}. \quad (120)$$

Combining the HP-OV transition with the MT-MQ bound proves the lemma. $\square$

Proof of Theorem 14. Lemma 15 gives

$$\text{AdvEUF-CMA}_{\text{Origami}}(\mathcal{A}) \leq \text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C}) + Q_s B_{\text{bad}} + (Q_h + Q_s) Q_s 2^{-\lambda_{\text{salt}}}. \quad (121)$$

Lemma 17 gives

$$\text{AdvEUF-KOA}_{\text{Origami}}(\mathcal{C}) \leq \text{AdvHP-OV}_{\text{Origami}}(\mathcal{B}) + \text{AdvMT-MQ}_{N,M,q}(\mathcal{B}') + q^{-M}. \quad (122)$$

Substituting the second inequality into the first yields the claimed advantage bound in Theorem 14. The running-time bound follows because the simulations add only public-key expansion, random-oracle table management, target assignment, and public MQ evaluation, giving overhead $(Q_s + Q_h + 1) \cdot \text{poly}(N, M, \log q)$. $\square$

4.3 Discussion of the Advantage Loss in the Security Proof

The reduction contains three explicit loss terms. The term $Q_s B_{\text{bad}}$ comes from replacing sampling over $\text{GoodSig}_{P_{\text{pub}}}$ by uniform sampling over $\mathbb{F}_q^N$. The term $(Q_h + Q_s)Q_s 2^{-\lambda_{\text{salt}}}$ bounds collisions between freshly sampled target-domain inputs $\text{target} \parallel h_\mu \parallel \text{salt}$ and already programmed random-oracle inputs. The term q^{-M} corresponds to a forgery at a target-domain point never queried by the adversary; in that case the target is fresh uniform in $\mathbb{F}_q^M$.

The quantities B_{Origami} and B_{bad} have different roles. The former bounds the probability that one ideal signing attempt restarts because some local matrix $A_j(X_j)$ is not full row rank. It justifies the conditioned repeat-until-success signing distribution and is not added as a separate theorem loss. The latter bounds the density of public signature vectors outside $\text{GoodSig}_{P_{\text{pub}}}$, namely vectors that do not correspond to internal assignments satisfying all local full-row-rank conditions.

Therefore, under Assumptions 5, 7, 8, and 9, and when the explicit loss terms are sufficiently small for the selected security level, the EUF-CMA security of the compact-key Origami implementation reduces to HP-OV hidden-structure indistinguishability for the compact public-key distribution and MT-MQ inversion hardness over the eligible-affine random reference distribution in the target-derivation random-oracle model.

4.4 Analysis of Known Attacks

We now discuss known attack directions against the compact-key Origami implementation and provide concrete classical and quantum bit-complexity estimates for all submitted parameter instances. We classify the attacks according to the part of the security proof that they target. As in the reduction, the attacks are divided into three categories: attacks exploiting the explicit proof loss, attacks on the HP-OV hidden-structure problem, and attacks on the MT-MQ inversion problem. This classification allows us to separate generic loss-based attacks, hidden-structure recovery attacks, and direct public MQ inversion attacks, and to evaluate their costs for Origami-128, Origami-256, Origami-384, and Origami-512.

The attacker is assumed to know the compact public key $pk = \text{param_id} \parallel \text{seed}_{\text{pk}} \parallel R_{\text{pk}}$, the public residual-position set $I_{\text{res}} = \text{SelectRes}(\text{params}, \text{seed}_{\text{pk}}) \subseteq I_{\text{elig}}$, and the public coordinate permutation Π_{pub} . Thus, seed_{pk} , I_{res} , I_{elig} , and R_{pk} are public data. The relevant hidden-structure question is whether these public data, together with the expanded map $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$, allow an attacker to recover the private oil-coordinate partitions ρ_j , the sampled/solved split (U_j, W_j) , the local zone boundaries, the hidden local algebras, or the sequential solving order. Let $\text{HiddenCost}_{\text{Origami}}$ denote the cost of recovering exploitable hidden structure from the compact public description, and let $\text{InvertCost}_{\text{Origami}}$ denote the cost of inverting P_{pub} on target-derivation outputs. A conservative classical attack cost can be summarized as

$$\text{AttackCost}_{\text{Origami}} \geq \min \{ \text{LossCost}_{\text{Origami}}, \text{HiddenCost}_{\text{Origami}}, \text{InvertCost}_{\text{Origami}} \}. \quad (123)$$

Attacks exploiting the loss in the security proof.

The first class of attacks targets the explicit loss terms in the reduction. These attacks do not necessarily break the HP-OV or MT-MQ assumptions, but exploit the gap between

the real compact-key security game and the idealized games used in the proof.

1. **Target-derivation collisions and random-oracle programming collisions.** The target-domain input has the form $x = \text{target} \parallel h_\mu \parallel \text{salt}$, where $h_\mu = H_{\text{msg}}(\mu)$. If two such inputs collide, or if a freshly sampled signing input collides with an already programmed random-oracle input, an adversary may try to transfer a valid signature from one message-salt pair to another. In the proof, this effect is bounded by $(Q_h + Q_s)Q_s 2^{-\lambda_{\text{salt}}}$. Generic target-space collision search is also reflected by the term $q^{M/2}$ in the abstract cost bound.
2. **Secret-key guessing.** The compact secret key is the master secret expansion seed $sk = \text{seed}_{\text{sk}}$. The public expansion seed seed_{pk} , the residual vector R_{pk} , and the residual-position set I_{res} are public and are not counted as secret material. If seed_{sk} is sampled uniformly from κ_{sk} bits, then classical exhaustive search has average cost $2^{\kappa_{\text{sk}}-1}$.
3. **Fresh-target guessing.** An adversary may output a forgery at a target-domain input that it never queried. For $x^* = \text{target} \parallel H_{\text{msg}}(\mu^*) \parallel \text{salt}^*$, if x^* was never queried, then $\mathcal{H}(x^*)$ is fresh uniform in $\mathbb{F}_q^M$. Hence, for any fixed σ^* , $P_{\text{pub}}(\sigma^*) = \mathcal{H}(x^*)$ holds with probability at most q^{-M} . A generic attack relying only on fresh-target guessing therefore needs about q^M independent trials.
4. **Restart leakage.** A local resampling event occurs when $\text{rank}(A_j(X_j)) < M_j$. The signer then resamples the current-zone vinegar variables and the sampled part U_j , and tries the same local target block again. If rank failures were frequent or strongly correlated with hidden structure, the restart behavior could leak information about successful internal assignments. The proof controls this by the restart bound B_{Origami} , which is used to justify the conditioned repeat-until-success signing distribution. It is not introduced as an additional additive loss term in the theorem.
5. **Bad-set density.** The proof also uses B_{bad} to bound the density of public signature vectors outside $\text{GoodSig}_{P_{\text{pub}}}$. These vectors do not correspond to internal assignments satisfying all local full-row-rank conditions. This term measures the loss in the game hop from the conditioned signing distribution to uniform public-signature sampling, rather than a direct structural key-recovery attack.

Collecting these loss-based directions gives the abstract lower bound

$$\text{LossCost}_{\text{Origami}} \geq \min \{ q^{M/2}, 2^{\kappa_{\text{sk}}-1}, q^M, B_{\text{Origami}}^{-1}, B_{\text{bad}}^{-1} \}, \quad (124)$$

up to constant factors and model-dependent overheads. Here $q^{M/2}$ corresponds to generic target-space collision or claw-style search, $2^{\kappa_{\text{sk}}-1}$ to classical secret-seed search, q^M to fresh-target guessing, and B_{Origami}^{-1} and B_{bad}^{-1} to the regularity quantities used in the proof.

Attacks on the hidden-structure problem.

The second class of attacks attempts to recover exploitable signer-side structure from the compact public description and the expanded public map. This is the attack surface captured by the HP-OV assumption. Unlike a classical global UOV instance, compact-key Origami does not publish a single global oil space, a public sampled/solved template, or a visible local-zone decomposition. A successful attack must therefore convert the public

data $P_{\text{pub}}, I_{\text{res}} \subseteq I_{\text{elig}}, R_{\text{pk}}, \text{seed}_{\text{pk}}$, and Π_{pub} into useful information about $\rho_j, (U_j, W_j)$, local zone boundaries, hidden local algebras, or the solving order.

We summarize the hidden-structure attack surface as

$$\text{HiddenCost}_{\text{Origami}} \geq \min \left\{ \text{KSCost}_{\text{Origami}}, \text{RecCost}_{\text{Origami}}, \text{WedgeCost}_{\text{Origami}}, \right. \\ \left. \text{IntCost}_{\text{Origami}}, \text{MinRankCost}_{\text{Origami}} \right\}. \quad (125)$$

1. **Recovery of a global hidden decomposition.** An attacker may search for a low-dimensional hidden subspace or for a global oil-vinegar-like decomposition after a change of coordinates. For Origami, this is useful only if the expanded map $P_{\text{pub}} = P_{\text{seed}} + \text{Embed}_{I_{\text{res}}}(R_{\text{pk}})$ still admits a coherent global model. The intended obstacle is that the signer-side structure is local and sequential, involving hidden local algebras, private partitions ρ_j , sampled/solved splits (U_j, W_j) , and local zone ordering rather than one global oil space.
2. **Kipnis-Shamir-type attacks.** Kipnis-Shamir-type attacks try to exploit eigenvector structure or related linear-algebraic signatures of a hidden oil space. In compact-key Origami, such an attack would first have to recover a usable global hidden-space description from P_{pub} and the public compact-key data. If the attack enumerates $q^{g_{\text{KS}}}$ candidate descriptions and tests each at cost $C_{\text{KS}}^{\text{test}}$, then $\text{KSCost}_{\text{Origami}} \geq q^{g_{\text{KS}}} C_{\text{KS}}^{\text{test}}$.
3. **Reconciliation-type attacks.** Reconciliation-type attacks search for structured zeros, affine relations, or correlated solution subsets of P_{pub} . Such relations become useful only if they can be aligned with private objects such as $\rho_j, (U_j, W_j)$, local zone boundaries, or hidden local algebras. If p_{hid} denotes the probability that a sampled relation reveals useful hidden information after public consistency tests, then $\text{RecCost}_{\text{Origami}} \geq \text{SolveCost}_{N,M,q}/p_{\text{hid}}$.
4. **Wedge-product / exterior-algebra attacks.** Because the submitted implementation uses $q = 16$, wedge-product attacks in characteristic two must be considered. These attacks use polar forms and the Plücker embedding to recover hidden oil-space information in UOV-like settings. For Origami, the attack must be applied to the polar form $P'_{\text{pub}}(x, y) = P_{\text{pub}}(x + y) - P_{\text{pub}}(x) - P_{\text{pub}}(y)$ of the expanded public map, together with the public support information $I_{\text{res}} \subseteq I_{\text{elig}}, R_{\text{pk}}$, and Π_{pub} . It succeeds only if these data reveal a coherent global invariant corresponding to the private partitions, local zones, or hidden algebras. Its abstract contribution is denoted by $\text{WedgeCost}_{\text{Origami}}$.
5. **Intersection attacks.** Intersection attacks search for several correlated hidden vectors or structural witnesses and solve an induced algebraic instance involving their intersections. In Origami, these witnesses must consistently point to the same private structure across the expanded map and the public support information. If an attack uses tuples of size t , solves the induced instance at cost $\text{SolveCost}_{N,M,q}^{(t)}$, and succeeds with probability $p_{\text{int}}(t)$, then $\text{IntCost}_{\text{Origami}}(t) \geq \text{SolveCost}_{N,M,q}^{(t)}/p_{\text{int}}(t)$, with the attacker minimizing over admissible t .
6. **Recognition of public support structure.** An attacker may try to detect visible zone boundaries, repeated index patterns, correlated coefficient supports, or other

regularities in the public compact-key data. Since I_{res} , I_{elig} , R_{pk} , seed_{pk} , and Π_{pub} are already public, the relevant question is whether these data reveal a usable alignment with private zone boundaries, partitions ρ_j , or sampled/solved splits (U_j, W_j) . Such recognition is mainly a preprocessing step for stronger hidden-structure attacks.

7. **Recovery of hidden local algebras.** Origami uses hidden local algebras $K_{j,i}$ internally. An attacker may search for invariant subspaces, conjugation fingerprints, or algebraic relations indicating these local representations. Since the local algebras are not published, the attacker must infer them from P_{pub} , Π_{pub} , and the public coefficient-support data. If successful, this attack may reduce the eligible-affine public MQ instance to a structured hidden-partition instance.
8. **Low-rank and MinRank-type attacks.** The polar forms of the expanded public equations may reveal low-rank patterns, approximate low-rank profiles, or structured rank deficiencies. Such an attack becomes effective only if the detected rank structure can be converted into private information about ρ_j , (U_j, W_j) , local zone boundaries, or hidden local algebras.
9. **Rectangular MinRank-type attacks.** A more specific case is a rectangular MinRank attack exploiting unusually low-rank linear maps induced by public polar forms. In compact-key Origami, the key question is not whether local rectangular systems exist internally, but whether the expanded public map exposes a global rank-constrained instance useful for recovering private structure. A generic summary is $\text{MinRankCost}_{\text{Origami}} \geq \text{MinRankSolveCost}(N, M, r_{\text{hid}})$.

These hidden-structure attacks may interact. Support recognition or local-algebra recovery may reduce the cost of Kipnis-Shamir, reconciliation, wedge, intersection, or MinRank attacks. Conversely, if no such preprocessing succeeds, the attacker is left with the expanded eligible-affine MQ map and must either attack the HP-OV assumption or solve the MT-MQ inversion problem.

Attacks on the MT-MQ inversion problem.

The third class of attacks treats P_{pub} as an eligible-affine public MQ map and attempts to invert it on target-derivation outputs. This is the attack surface captured by the MT-MQ assumption after no usable hidden structure has been recovered.

1. **Direct attacks.** For a target-domain input $x = \text{target} \parallel h_\mu \parallel \text{salt}$, the adversary obtains $t = \mathcal{H}(x)$ and solves the public system $P_{\text{pub}}(\sigma) = t$. Possible methods include XL, Gröbner basis methods, hybrid guessing-and-elimination, and algorithms for underdetermined MQ systems. Abstractly, $\text{DirectCost}_{\text{Origami}} \geq \text{SolveCost}_{N,M,q}$.
2. **Hybrid attacks.** Hybrid attacks guess part of the public signature variables, target constraints, or auxiliary public variables, and then solve the remaining public system. If the attack guesses private partitions, sampled/solved splits, local boundaries, or hidden algebras, it belongs to the HP-OV layer instead. In the MT-MQ layer, guessing g public coordinates gives $\text{HybridCost}_{\text{Origami}}(g) \geq q^g \text{SolveCost}_{N-g,M,q}$.
3. **Multi-target attacks.** The adversary may enumerate many salts and obtain many targets $t_i = \mathcal{H}(\text{target} \parallel h_\mu \parallel \text{salt}_i)$. It may then search for a target that is easier to invert. This motivates the MT-MQ formulation: the attacker faces inversion over a sequence of independent targets rather than a single fixed target.

4. **Collision attacks.** Instead of solving a fixed preimage instance, the adversary may sample candidate signatures σ , compute $P_{\text{pub}}(\sigma)$, and compare these values with targets generated by the target-derivation mechanism. Such attacks are generic target-space attacks and do not recover private oil-coordinate partitions, sampled/solved splits, local boundaries, or hidden local algebras.
5. **Claw-finding attacks.** Claw-finding attacks search for a pair (σ, x) satisfying $P_{\text{pub}}(\sigma) = \mathcal{H}(x)$, where $x = \text{target} \parallel h_\mu \parallel \text{salt}$. If the attacker computes X public-map values and Y target-derivation values, the expected number of matches is about XY/q^M . Constant success requires $XY \approx q^M$, and minimizing $C_P X + C_H Y$ gives $X = \sqrt{C_H q^M / C_P}$ and $Y = \sqrt{C_P q^M / C_H}$. Hence

$$\text{ClawCost}_{\text{Origami}} \geq 2\sqrt{C_P C_H q^M}. \quad (126)$$

Collecting the MT-MQ-layer attacks gives

$$\text{InvertCost}_{\text{Origami}} \geq \min \left\{ \text{SolveCost}_{N,M,q}, \min_g q^g \text{SolveCost}_{N-g,M,q}, 2\sqrt{C_P C_H q^M} \right\}. \quad (127)$$

The design goal is therefore not to reduce every generic algebraic attack on the expanded public system, but to remove structural shortcuts that would reveal the private oil-coordinate partitions, sampled/solved splits, local zone boundaries, hidden local algebras, or sequential solving order.

Concrete complexity estimates for known attacks.

We now give concrete bit-complexity estimates for the known classical and quantum attack directions considered above. The submitted Origami instances are defined over F_{16} , and hence $q = 16 = 2^4$. Therefore, for a public target space of dimension M , one has $q^M = 2^{4M}$. The estimates below are stated as work factors up to polynomial factors in public-map evaluation, random-oracle evaluation, and finite-field arithmetic.

For the loss-based generic attacks, we use the following estimates. Classical exhaustive search over the compact secret key has cost

$$C_{\text{seed}}^{\text{cl}} = 2^{\kappa_{\text{sk}}}. \quad (128)$$

A classical target-space claw-finding attack has generic cost

$$C_{\text{claw}}^{\text{cl}} = q^{M/2} = 2^{2M}. \quad (129)$$

A fresh-target preimage search attack has classical cost

$$C_{\text{pre}}^{\text{cl}} = q^M = 2^{4M}. \quad (130)$$

For quantum attacks, Grover search reduces secret-key exhaustive search to

$$C_{\text{seed}}^{\text{q}} = 2^{\kappa_{\text{sk}}/2}. \quad (131)$$

Table 11 – Known classical attack complexity of Origami instances.

Scheme	(N, M)	Classical security target	Secret-key exhaustive search	Target-space claw-finding	Fresh-target preimage search	Minimum known classical work factor
Origami-128	(200, 104)	2^{128}	2^{128}	2^{208}	2^{416}	$\geq 2^{128}$
Origami-256	(968, 488)	2^{256}	2^{256}	2^{976}	2^{1952}	$\geq 2^{256}$
Origami-384	(1800, 904)	2^{384}	2^{384}	2^{1808}	2^{3616}	$\geq 2^{384}$
Origami-512	(2312, 1160)	2^{512}	2^{512}	2^{2320}	2^{4640}	$\geq 2^{512}$

Table 12 – Known quantum attack complexity of Origami instances.

Scheme	(N, M)	Quantum security target	Grover search over the secret key	Quantum target-space claw-finding	Grover search for fresh-target preimages	Minimum known quantum work factor
Origami-128	(200, 104)	2^{64}	2^{64}	$2^{138.7}$	2^{208}	$\geq 2^{64}$
Origami-256	(968, 488)	2^{128}	2^{128}	$2^{650.7}$	2^{976}	$\geq 2^{128}$
Origami-384	(1800, 904)	2^{192}	2^{192}	$2^{1205.3}$	2^{1808}	$\geq 2^{192}$
Origami-512	(2312, 1160)	2^{256}	2^{256}	$2^{1546.7}$	2^{2320}	$\geq 2^{256}$

It also reduces fresh-target preimage search to

$$C_{\text{pre}}^q = q^{M/2} = 2^{2M}. \quad (132)$$

For generic quantum target-space claw-finding attacks, the standard quantum claw-finding estimate gives

$$C_{\text{claw}}^q \approx q^{M/3} = 2^{4M/3}. \quad (133)$$

These estimates quantify the generic target-space attacks and secret-key search attacks whose costs can be directly evaluated from κ_{sk} and M . The hidden-structure attacks and direct MQ inversion attacks are discussed separately after the tables, because their costs depend on whether the public compact-key description reveals exploitable algebraic structure.

Table 11 gives the classical complexities of the known generic attack directions for all submitted instances. For Origami-128, Origami-256, Origami-384, and Origami-512, the smallest explicit generic classical attack is secret-key exhaustive search, with costs 2^{128} , 2^{256} , 2^{384} , and 2^{512} , respectively. The target-space attacks are much larger because the public target dimension M is large for all submitted instances.

The hidden-structure attacks considered above, including Kipnis-Shamir-type attacks, reconciliation attacks, wedge-product or exterior-algebra attacks, intersection attacks, support-recognition attacks, hidden-local-algebra recovery attacks, and MinRank-type attacks, must recover exploitable information about the private oil-coordinate partitions, the sampled/solved split, the local zone boundaries, the hidden local algebras, or the sequential solving order from the compact public description. At present, no known attack in this class gives a concrete classical work factor below the classical security target for the submitted parameter sets. Similarly, the known direct and hybrid MQ inversion attacks against the expanded eligible-affine public MQ map do not give a concrete classical work factor below the classical security target. Therefore, among the known classical attack directions considered in this work, the minimum work factor for each instance remains bounded below by the corresponding classical security target shown in Table 11.

Table 12 gives the corresponding quantum complexities. The smallest known quantum attack direction is Grover search over the compact secret key. This gives 2^{64} , 2^{128} ,

2^{192} , and 2^{256} operations for Origami-128, Origami-256, Origami-384, and Origami-512, respectively. Generic quantum target-space claw-finding attacks and Grover search for fresh-target preimages are far above these levels. For hidden-structure recovery and direct MQ inversion, no known quantum attack gives a concrete work factor below the quantum security target for the submitted instances.

The quantum estimates above cover the known generic quantum speedups relevant to the attack classes considered here, including Grover search and quantum target-space claw-finding. They do not constitute a full quantum random-oracle-model proof. A complete QROM treatment would require quantum versions of the HP-OV hidden-structure indistinguishability assumption and the MT-MQ multi-target inversion assumption.

Consequently, the submitted parameter sets provide the following concrete security levels against the known classical and quantum attack directions considered in this work: Origami-128 provides 2^{128} classical and 2^{64} quantum security; Origami-256 provides 2^{256} classical and 2^{128} quantum security; Origami-384 provides 2^{384} classical and 2^{192} quantum security; and Origami-512 provides 2^{512} classical and 2^{256} quantum security.

Quantum attacks.

The present proof is classical and is stated in the classical target-derivation random-oracle model. A quantum treatment would require a QROM proof and quantum versions of the HP-OV and MT-MQ assumptions. We nevertheless record how the attack classes above may be affected by quantum speedups.

1. **Target-derivation collision and claw-finding attacks.** Quantum collision or claw-search techniques may accelerate target-space matching between P_{pub} and the target-derivation mechanism. The actual gain depends on the cost of coherent evaluations, the structure of the search space, and whether the attack can be implemented as a suitable quantum search routine.
2. **Direct system-solving attacks.** For direct and hybrid attacks on $P_{\text{pub}}(\sigma) = t$, quantum search may speed up guessing or enumeration phases. It does not automatically reduce the full Gröbner, XL, linear-algebra, or verification cost by a square-root factor unless a genuinely faster quantum MQ-solving method applies to the eligible-affine public systems.
3. **Structure-recovery attacks.** For Kipnis-Shamir, reconciliation, wedge-product, intersection, and MinRank-type attacks, quantum search may accelerate candidate enumeration. However, each candidate must still be verified against P_{pub} and the public compact-key data, and then converted into useful information about ρ_j , (U_j, W_j) , local zones, hidden algebras, or the solving order.
4. **Secret-key guessing and fresh-target guessing.** These are the most direct quantum-search targets. Grover search reduces exhaustive search over $sk = \text{seed}_{\text{sk}}$ from about $2^{\kappa_{\text{sk}}}$ to about $2^{\kappa_{\text{sk}}/2}$, and reduces fresh-target search from about q^M to about $q^{M/2}$. Concrete parameter choices must therefore leave sufficient margin against both forms of quantum search.

In summary, the current discussion does not identify a separate quantum-only structural weakness. Quantum attacks are treated as parameter-selection constraints rather than as a theorem proved by the present classical reduction. A complete quantum claim

Table 13 – Estimated signing failure probabilities of the submitted Origami instances.

Scheme	Classical security	Quantum security	Local zones	P_{abort}
Origami-128	128-bit	64-bit	4	$< 2^{-33482}$
Origami-256	256-bit	128-bit	7	$< 2^{-34481}$
Origami-384	384-bit	192-bit	9	$< 2^{-45245}$
Origami-512	512-bit	256-bit	10	$< 2^{-40581}$

would require a dedicated QROM proof and quantum analyses of the HP-OV and MT-MQ assumptions.

4.5 Signing Failure Probability and Its Security Impact

This subsection analyzes the signing failure probability of Origami and its effect on correctness and security. The signing algorithm may encounter local rank-deficiency events during signature generation. Such an event occurs when the local matrix in one signing zone does not have full row rank. In this case, the signer discards the current local randomness and repeats the local sampling procedure. Therefore, this event is a local restart event rather than an invalid-signature event.

For local zone j , let β_j denote the probability that one evaluated local matrix does not have full row rank. If the implementation allows at most $R_{\max}$ attempts for each local zone, then local zone j aborts only when all $R_{\max}$ attempts fail. Hence, the abort probability of local zone j is bounded by

$$P_{\text{abort},j} \leq \beta_j^{R_{\max}}. \quad (134)$$

Since the full signing algorithm aborts only if at least one local zone aborts, the overall signing abort probability satisfies

$$P_{\text{abort}} \leq 1 - \prod_{j=1}^{d+1} (1 - \beta_j^{R_{\max}}) \leq \sum_{j=1}^{d+1} \beta_j^{R_{\max}}. \quad (135)$$

This bound shows that the final signing failure probability is exponentially smaller than the probability of one local restart.

In the submitted implementation, the per-zone attempt bound is set to $R_{\max} = 8192$. Using the measured local restart behavior of the submitted parameter sets, the estimated final signing failure probabilities are shown in Table 13. These probabilities refer to the final signing abort event after all allowed attempts have failed. They should not be interpreted as the probability of one local restart.

The values in Table 13 are far below the claimed classical and quantum security targets. Thus, the probability that the signing algorithm finally aborts is negligible for all submitted Origami instances. The local restart behavior may increase the signing time, but it does not affect the validity of an output signature.

We now state this property formally.

Theorem 18 (Signing failure probability) *Let $R_{\max}$ be the maximum number of allowed attempts for each local zone. For each local zone $j \in \{1, \dots, d+1\}$, let β_j be an*

upper bound on the probability that one evaluated local matrix is rank-deficient. Then the final signing abort probability satisfies

$$P_{\text{abort}} \leq 1 - \prod_{j=1}^{d+1} (1 - \beta_j^{R_{\max}}) \leq \sum_{j=1}^{d+1} \beta_j^{R_{\max}}. \quad (136)$$

Moreover, every successfully generated signature satisfies the public verification equation. Therefore, rank-deficiency events only affect the restart behavior and the implementation-level abort probability, but they do not produce invalid signatures.

Proof. For local zone j , once the prefix variables, the current-zone vinegar variables, and the sampled oil coordinates have been fixed, the signing equation is reduced to a local linear system in the solved coordinates. If the evaluated local matrix has full row rank, then the corresponding linear map is surjective onto the local target space. Hence, for every local target block, the local system has at least one solution.

Therefore, the only local failure event is that the evaluated local matrix does not have full row rank. By the definition of β_j , one local attempt in zone j fails with probability at most β_j . If the implementation allows at most $R_{\max}$ independent attempts for this zone, then zone j aborts only if all $R_{\max}$ attempts fail. Thus,

$$P_{\text{abort},j} \leq \beta_j^{R_{\max}}. \quad (137)$$

The full signing algorithm aborts only if at least one local zone aborts. Hence,

$$P_{\text{abort}} \leq 1 - \prod_{j=1}^{d+1} (1 - \beta_j^{R_{\max}}). \quad (138)$$

Applying the union bound gives

$$1 - \prod_{j=1}^{d+1} (1 - \beta_j^{R_{\max}}) \leq \sum_{j=1}^{d+1} \beta_j^{R_{\max}}. \quad (139)$$

If the signing algorithm terminates successfully, then every local zone has been solved successfully. Therefore, each local output block equals the corresponding target block. After flattening and applying the public coordinate convention, the resulting signature satisfies the public verification equation. Hence, a local rank-deficiency event can only cause resampling or a generic signing abort under the finite-attempt implementation; it cannot produce an invalid signature. $\square$

The security impact of signing failure is limited. In the ideal repeat-until-success specification, failed local attempts are discarded, and the output distribution is the distribution of successful attempts conditioned on all local full-rank events. In the finite-attempt implementation, if an adversary makes at most Q_s signing queries, the implementation-level abort event contributes at most the additional statistical term

$$Q_s P_{\text{abort}}. \quad (140)$$

Since the values of P_{abort} in Table 13 are negligible, this term does not materially affect the claimed security levels.

The implementation should also ensure that failed local attempts do not reveal partial information. In particular, it should not output the failed local-zone index, the retry count, intermediate local matrices, or partial assignments. The external interface should return only a valid signature or a generic signing failure. Under this convention, signing failure affects only availability and signing time, while every successfully generated signature remains correct and the restart behavior does not provide an additional algebraic object for hidden-structure recovery.